



北京航空航天大学  
BEIHANG UNIVERSITY

# 机器人导航python实践(入门)

## Lecture1-基础环境配置和准备

---

北航 国新院 实验实践课  
智能系统与人形机器人国际研究中心



教师： 欧 阳 老 师



邮 箱： ouyangkid@buaa.edu.cn



学 期： 2025年秋季

# 目录

## Contents

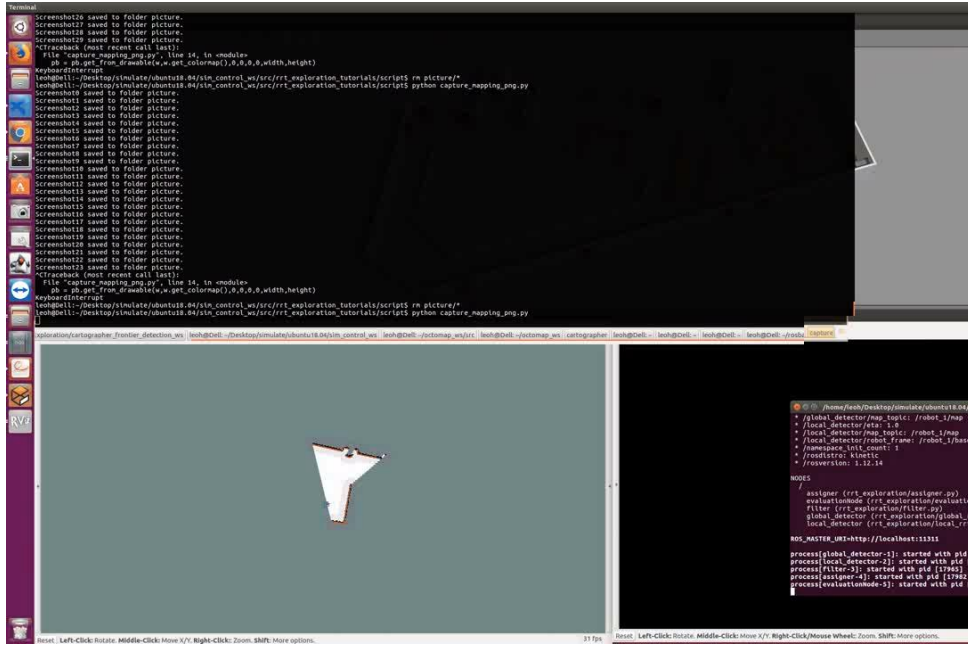
01 课程内容安排

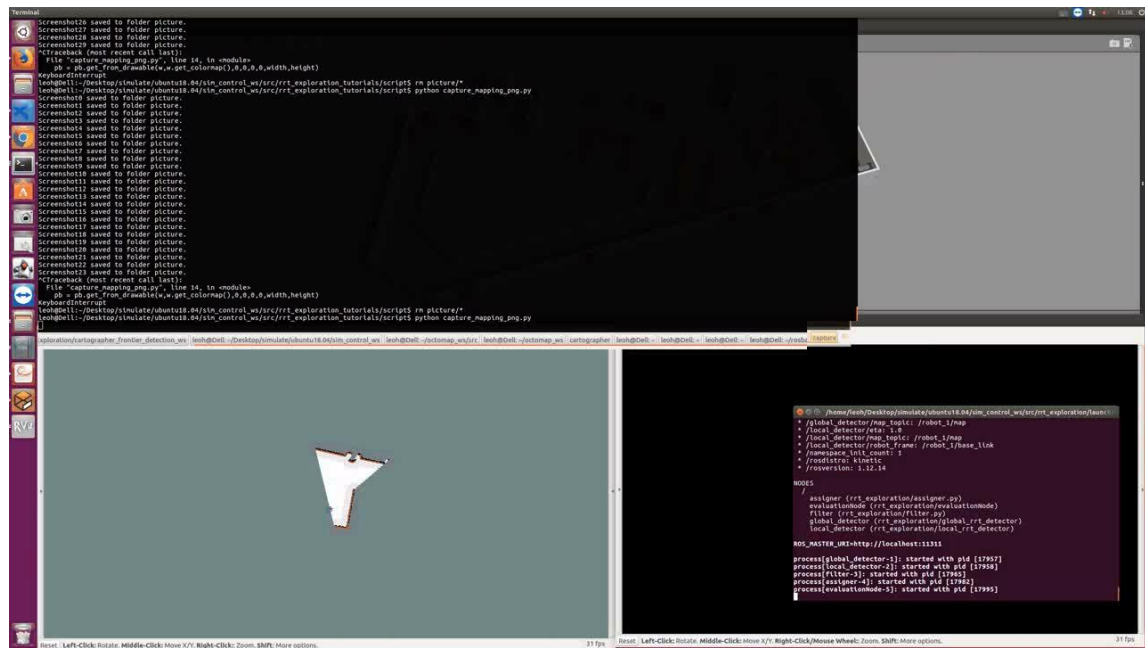
02 课程目标

03 学习目标

04 机器人简介

05 软件环境准备

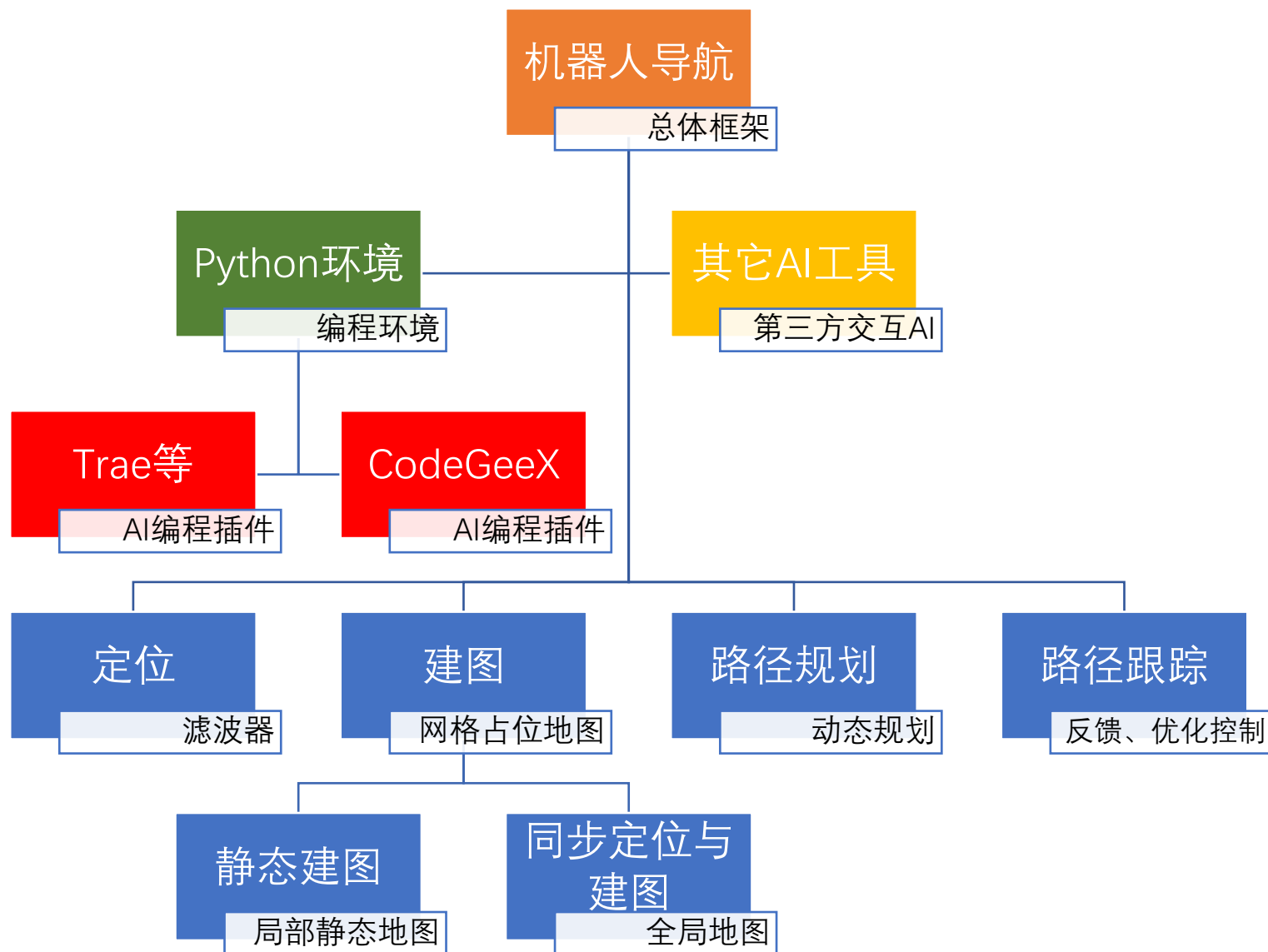
- 课程围绕**机器人导航**的理论和可视化实践的**5个核心内容**开展，具体包括
    - 环境基础（准备工作）：**4学时**
    - 机器人自主导航框架&定位：**4学时**
    - 建图（静态）：**4学时**
    - 同步定位与建图（动态）：**8学时**
    - 路径规划：**8学时**
    - 路径跟踪：**4学时**
- 
- The screenshot displays a ROS2 environment. The top terminal window shows repeated messages: "Screenshots saved to folder picture." and "KeyboardInterrupt". The bottom window shows a 3D simulation of a robot in a corridor, with a terminal on the right displaying ROS2 logs for the 'global\_detector' node.



# Part 1 | 课程内容安排

课程内容

## » 内容安排



# 目录

## Contents

01 课程内容安排

02 课程目标

03 学习目标

04 机器人简介

05 软件环境准备

# Part 2 | 课程目标



## 机器人导航python实践

### 目标学生

- 电子信息、交通运输、机械电子

### 学习内容

- 基于python的实践编程学习



#### 导航开发理念

机器人研究发展历史、关键组件及其功能，典型机器人类型。



- 了解机器人发展背景
- 导航在机器人技术栈中的位置
- 硬件的依赖
- 最小软件依赖



#### 自主导航系统框架

定位、建图、SLAM、路径规划和路径跟踪五大模块



- 自主导航系统的框架
- 各子模块的功能
- 模块间的耦合关联闭环



#### 算法理论

理解各核心模块的算法，结合可视化分析，掌握不同算法的优缺点和适应性。



- 算法依赖的数据及数据结构
- 算法类型、优势及效率评估
- 优化、控制基础



#### 编程实践

Python编程实现算法，利用可视化辅助理解，借助AI编程助手提高编程效率。



- 基础环境
- 基础依赖库使用
- 可视化解理解
- 交互AI辅助的开发流程

# 目录

## Contents

01 课程内容安排

02 课程目标

03 学习目标

04 机器人简介

05 软件环境准备

# Part 3 | 学习目标

## » 机器人导航python实践

### 1.系统-模块（架构）

- 自主导航各模块协同关系，理解从环境感知到轨迹追踪的完整流程逻辑。

### 2.核心算法（理论）

- 机器人环境感知的**数据结构及算法**
- 矩阵、凸优化
- 算法评估分析

### 3.软件系统（操作实践）

- 机器人模块：传感器、执行器、计算单元、能源、软件&算法
- 集成开发环境使用
- AI模型交互





# 目录

## Contents

01 课程内容安排

02 课程目标

03 学习目标

04 机器人简介

05 软件环境准备

# Part 4 | 机器人简介

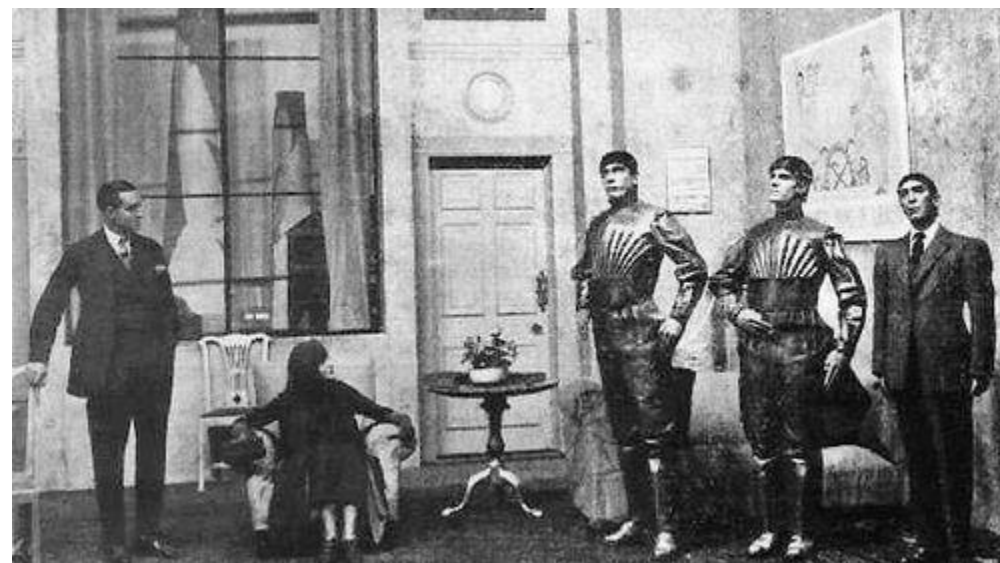
## Intorduction

### » Robot起源

**机器人**（英语：Robot，香港称为机械人）包括一切模拟人类行为或思想与模拟其他生物的机械（如机器狗、机器猫等）。--wikipedi

**狭义上**对机器人的定义还有很多分类法及争议，有些电脑程序甚至也被称为机器人。在当代工业中，机器人指能自动执行任务的人造机器设备，用以**取代或协助人类工作**，一般会机电设备，由计算机程序或是电子电路控制。

“机器人”源自捷克语的Robot一词，而捷克语的Robot一词最早出现在公元1920年捷克科幻作家恰配克的《罗索姆的万能机器人》中，原文作“Robota”，在捷克文当中指奴隶制，或者艰辛的劳役，后来成为西文中通行的“Robot”。

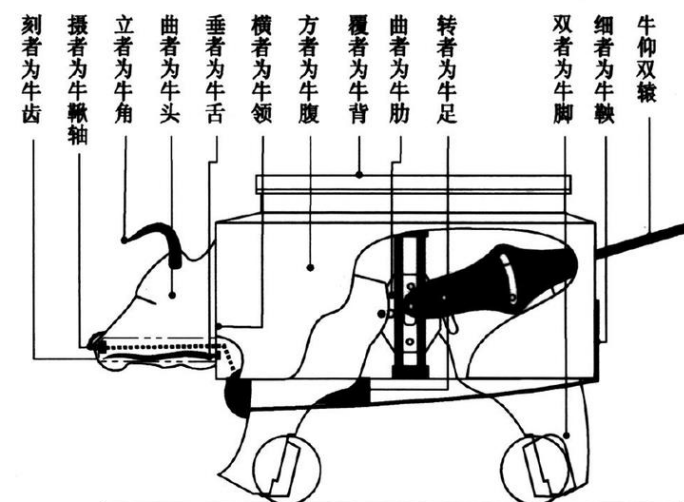


# Part 4 | 机器人简介

## Robot起源

## Intorduction

### 古代机器人



木牛流马（概念图）

塔罗斯Talos



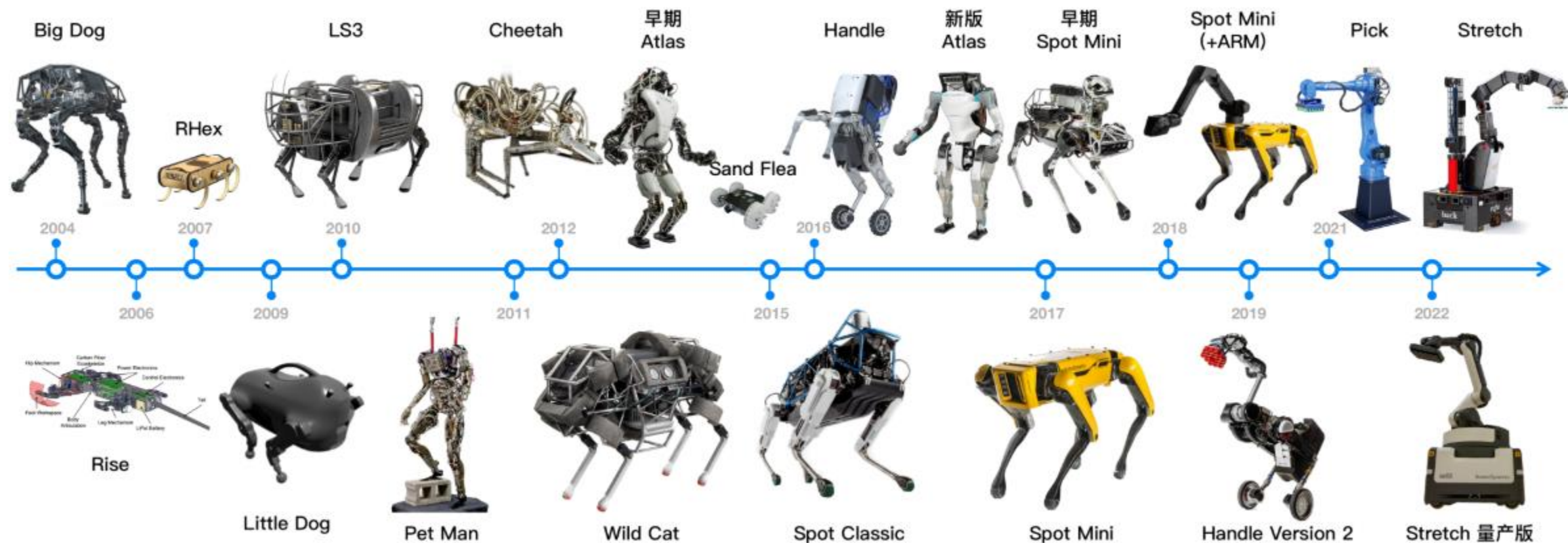


# Part 4 | 机器人简介

## Introduction

### Robot起源

**机器人学**（英语：robotics）涵盖了机器人的设计、建造、运作、以及应用的跨领域科技，集合机械工程学、电机工程学、机械电子学、电子学、控制工程学、**计算机**、**软件**、通讯、数学及生物工程学等领域。



轮式机器人-作战机器人

# Part 4 | 机器人简介

## Intorduction

### Robot起源

机器人学（英语：robotics）涵盖了机器人的设计、建造、运作、以及应用的跨领域科技，集合机械工程学、电机工程学、机械电子学、电子学、控制工程学、**计算机、软件**、通讯、数学及生物工程学等领域。



NOETIC  
2020年  
第13版ROS



KILLED KAIJU  
2024年  
ROS2



We support building ROS 2 from source o

- Ubuntu Linux 24.04
- Windows 10
- RHEL-9/Fedora
- macOS

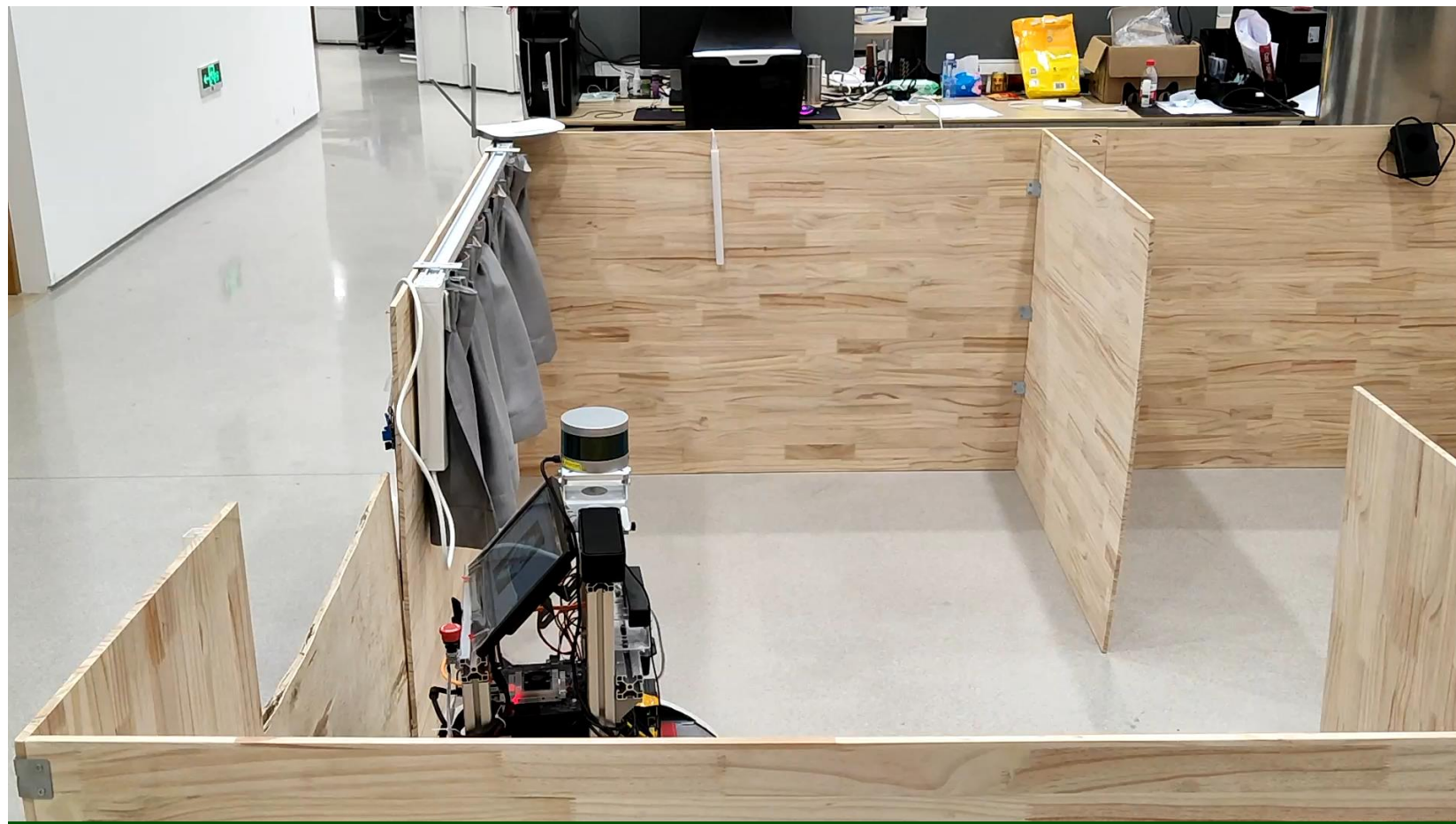
ROS Noetic Ninjemys is primarily targeted at the Ubuntu 20.04 (Focal) release, though other systems are supported to varying degrees. For more information on compatibility on other platforms, please see [REP 3: Target Platforms](#).

ROS  
Robot Operating System

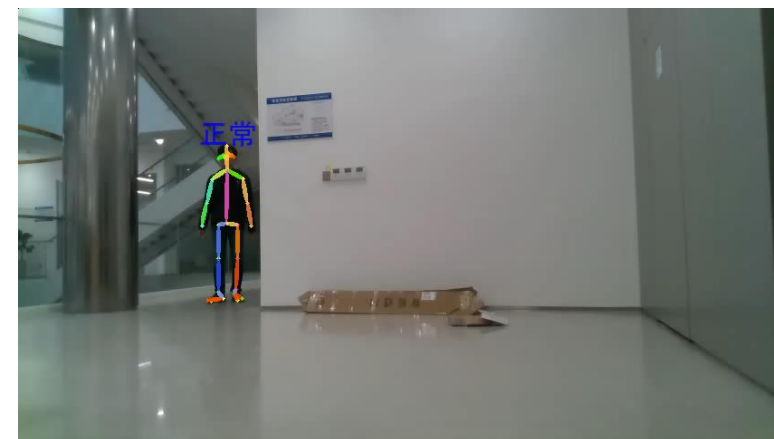
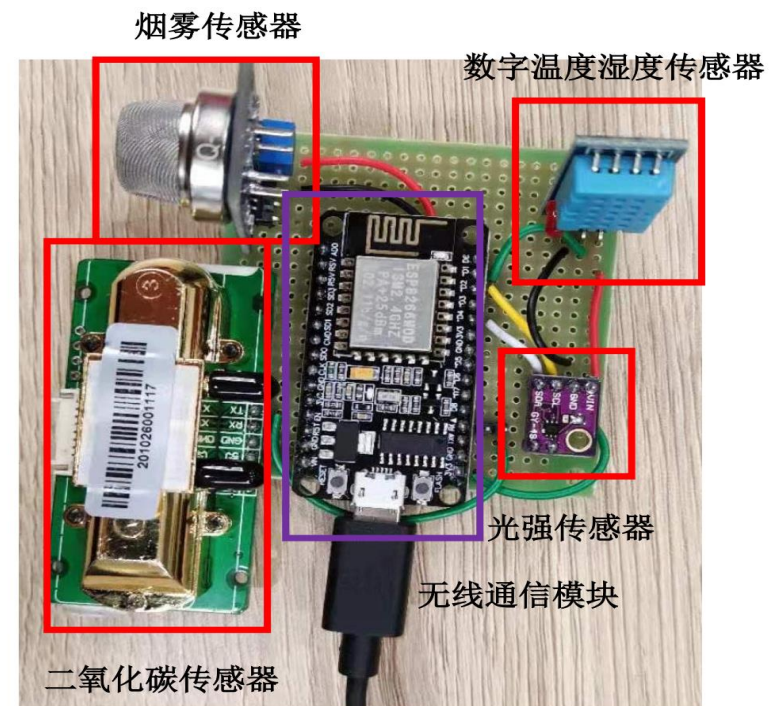


# Part 4 | 机器人简介

## 室内养老机器人



## Intorduction

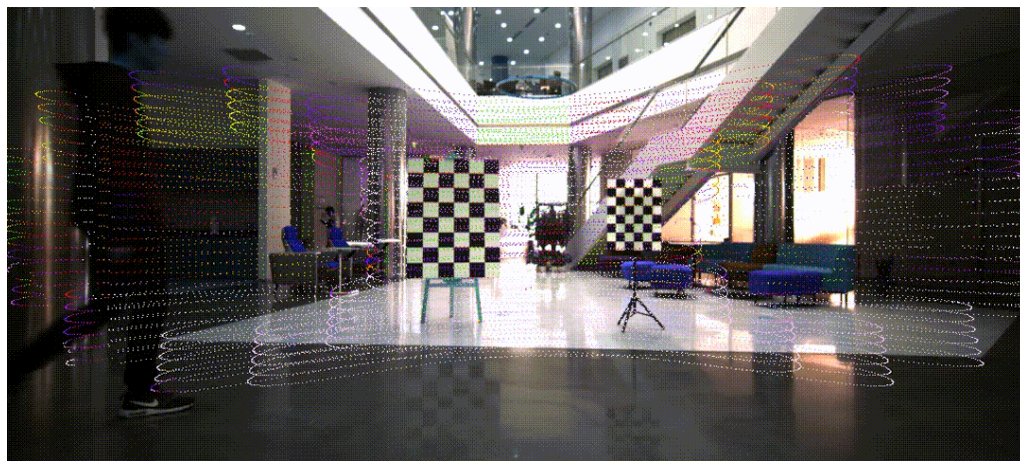
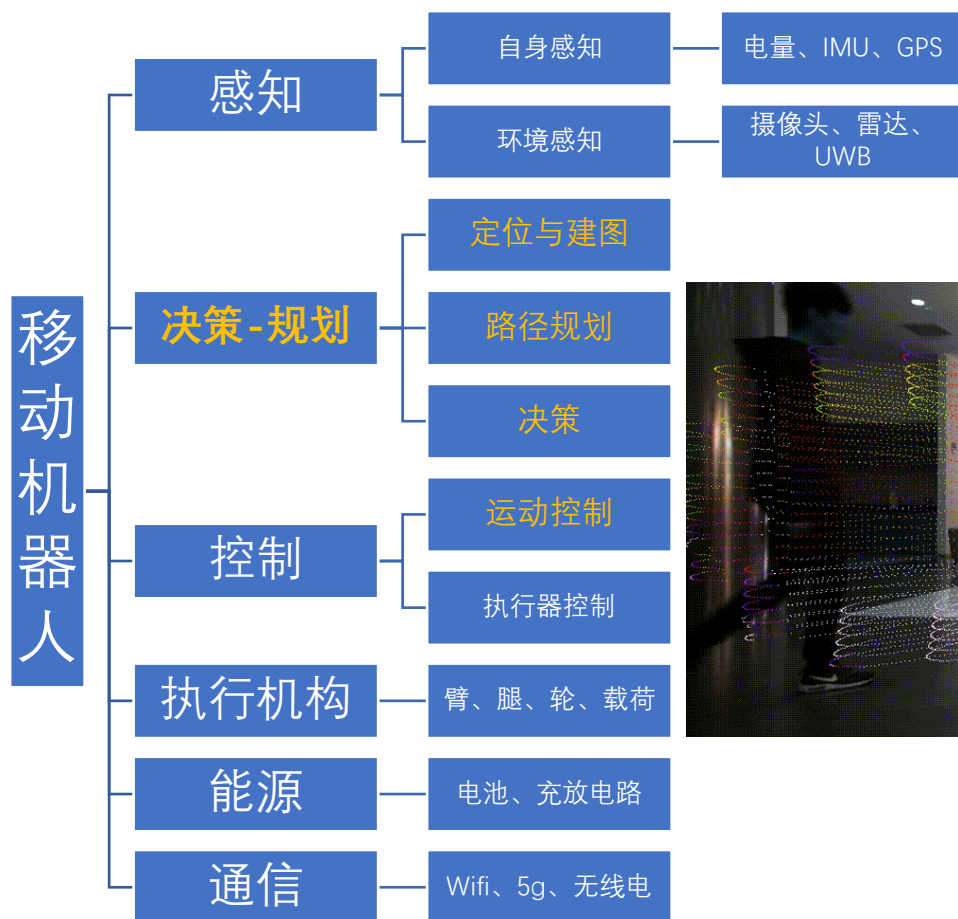


# Part 4 | 机器人简介

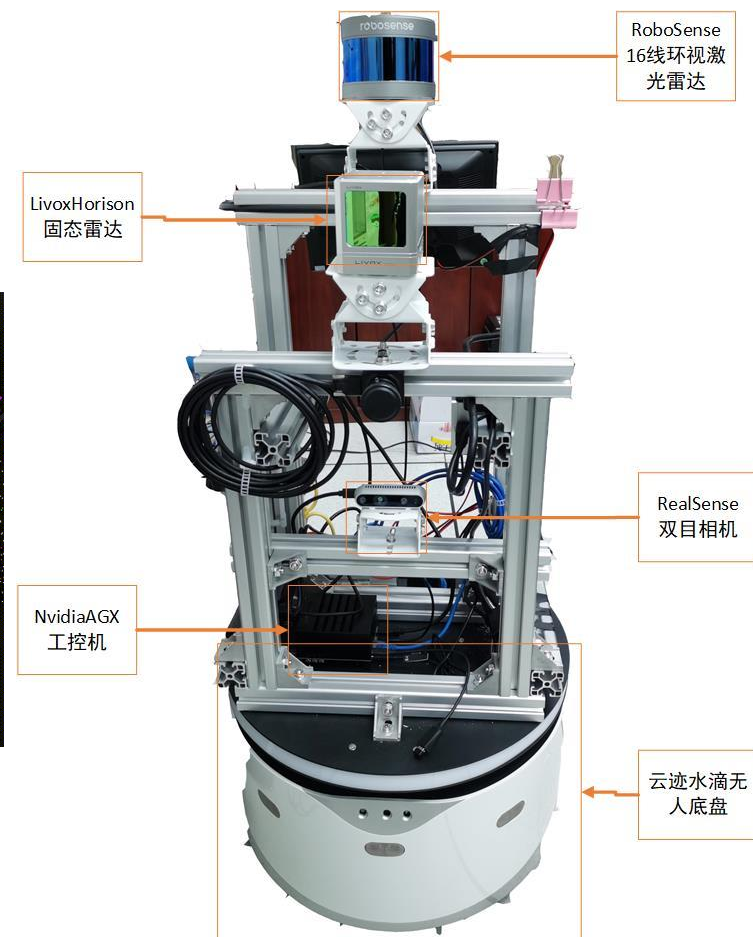
## Robot起源

## Intorduction

**机器人学**（英语：robotics）涵盖了机器人的设计、建造、运作、以及应用的跨领域科技，集合机械工程学、电机工程学、机械电子学、电子学、控制工程学、**计算机**、**软件**、通讯、数学及生物工程学等领域。



相机和雷达感知结果



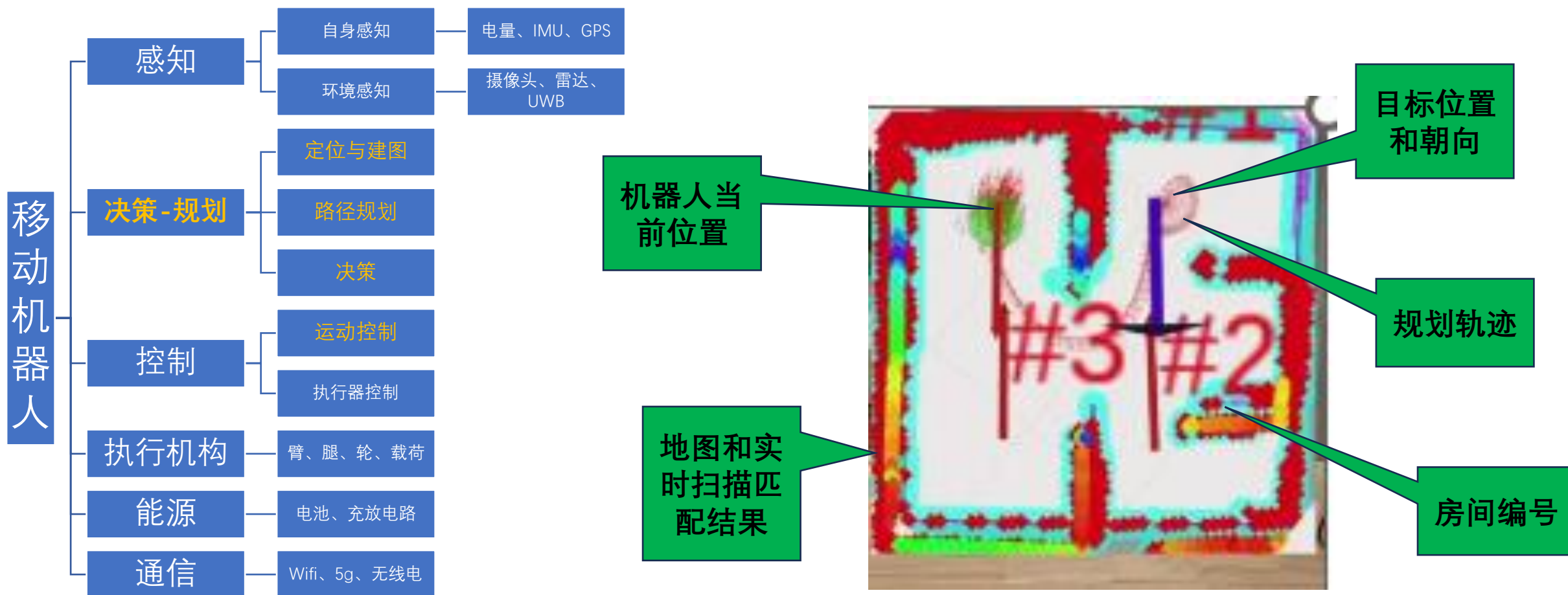


# Part 4 | 机器人简介

## Introduction

### Robot起源

**机器人学**（英语：robotics）涵盖了机器人的设计、建造、运作、以及应用的跨领域科技，集合机械工程学、电机工程学、机械电子学、电子学、控制工程学、**计算机**、**软件**、通讯、数学及生物工程学等领域。



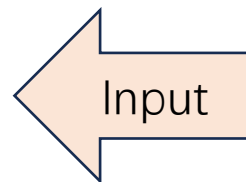
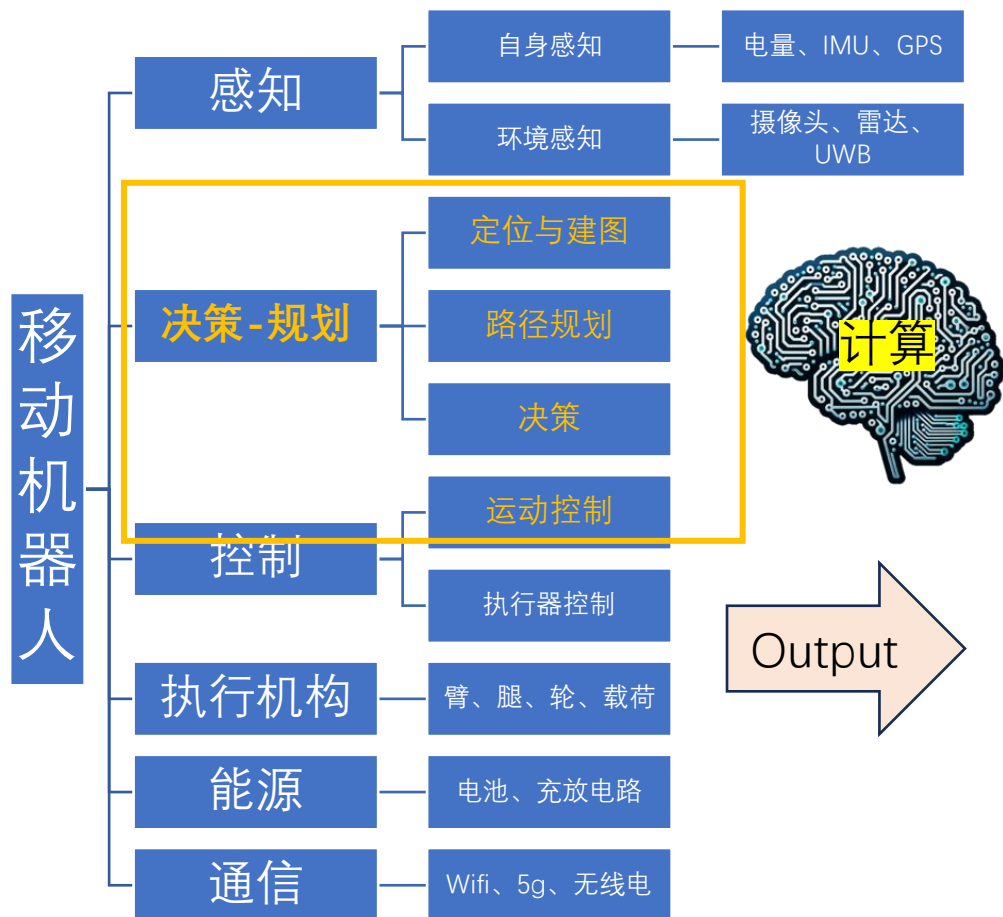


# Part 4 | 机器人简介

## Introduction

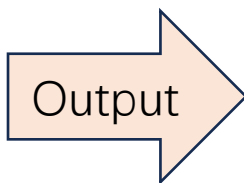
### Robot起源

**机器人学**（英语：robotics）涵盖了机器人的设计、建造、运作、以及应用的跨领域科技，集合机械工程学、电机工程学、机械电子学、电子学、控制工程学、**计算机**、**软件**、通讯、数学及生物工程学等领域。



①对感知输入进行简化和抽象

- 重点关注“决策-规划-控制”过程的核心数据结构、抽象算法；
- 利用可视化工具提升对算法的直观可理解性；
- 使用python和尽可能少的依赖库降低编程实践难度；
- 引入AI辅助，提高学习效率、降低门槛和交互难度。



- ②将机器人简化为质点，运动约束规约为简单二维李群代数
- ③忽略复杂机构动力学、电源和通信

# 目录

## Contents

01 课程内容安排

02 课程目标

03 学习目标

04 机器人简介

05 软件环境准备

# Part 5 | 软件环境准备

## Python环境

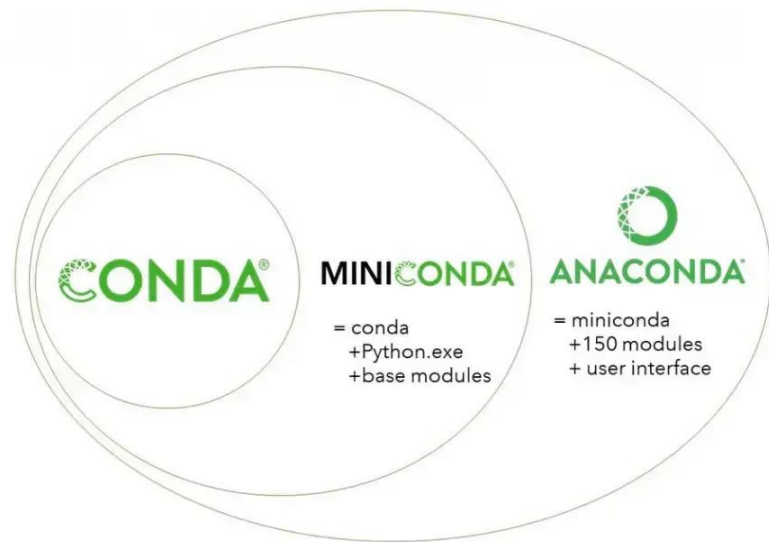
Pycharm 集成开发环境 + anaconda/miniconda



集成开发环境



Python环境管理



科学计算依赖库:

[NumPy](#) 基础科学计算库.

[SciPy](#) 科学和工程计算库.

[Matplotlib](#) 可视化绘图库.

[Pandas](#) 数值分析库.

[SymPy](#) 符号数学库 **sym** bolic mathematics.

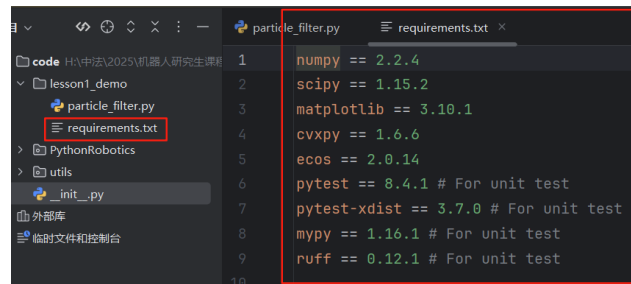
[CVXPY](#) 凸优化库.

功能库:

[Scikit-learn](#) 机器学习库.

[Scikit-image](#) 图像处理库.

[Networkx](#) 复杂网络 (图) 计算库.



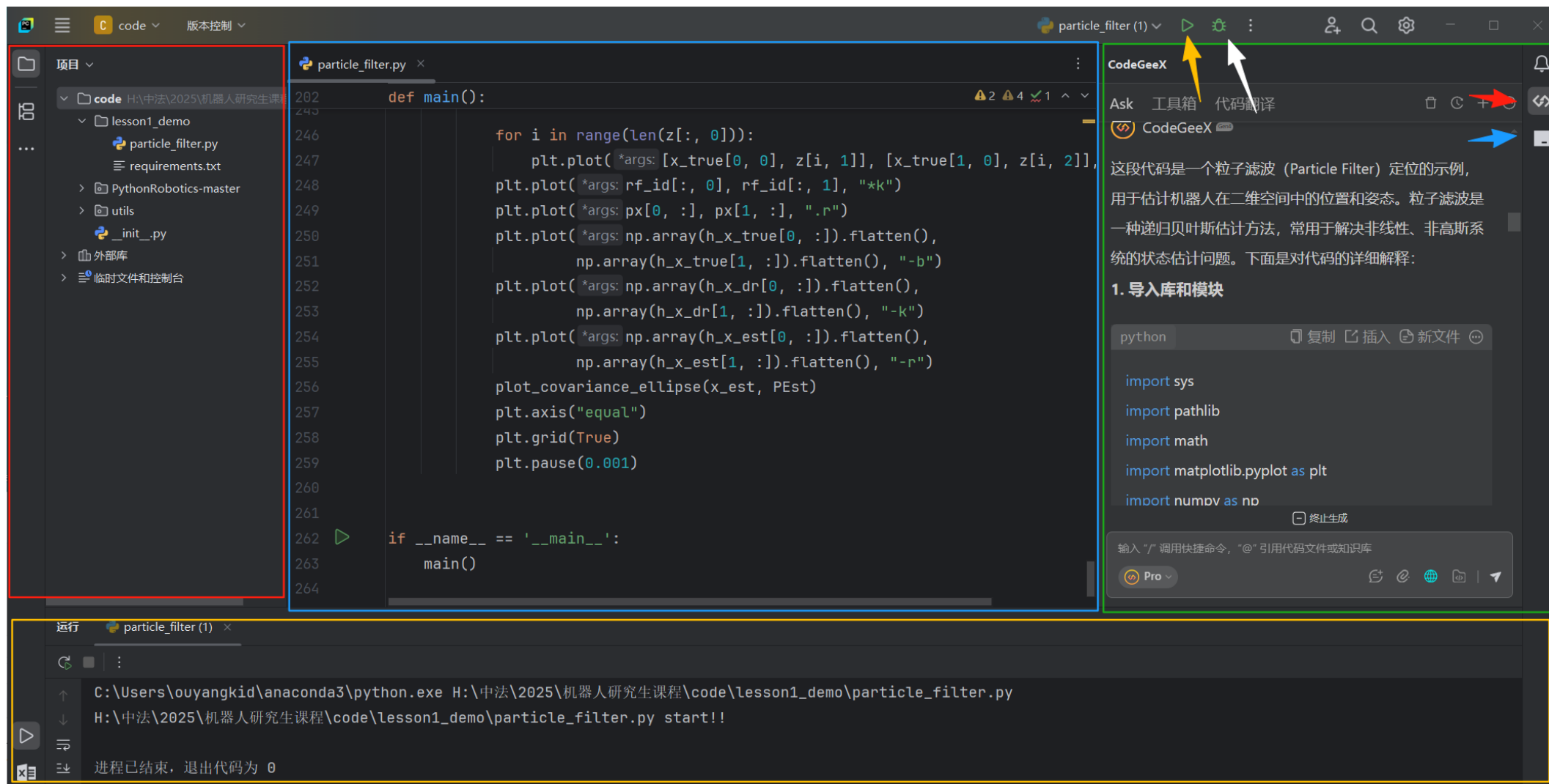
安装便捷

尽量安装**最新版本**软件，之前有学生上课因为以前安装的**老版本**，无法安装插件。

# Part 5 | 软件环境准备

## Python环境

### Pycharm 集成开发环境 + anaconda/miniconda



尽量安装最新版本软件，之前有学生上课因为以前安装的未升级，无法更新插件。

# Part 5 | 软件环境准备

## AI插件使用

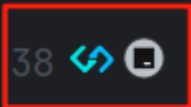
思辨习惯：永远不要只相信某一款大模型，一定要多模型交叉验证，以传统媒介为基准（Baseline）。

在对算法本身不了解的情况下，可通过与AI交互助手进行对话，获取关于相关算法的背景信息：

如，选择与Trae进行交互

```
37
38
39
40
41
42
def calc_input():
    v = 1.0 # [m/s]
    yaw_rate = 0.1 # [rad/s]
    u = np.array([[v, yaw_rate]]).T
    return u
```

解释 | 添加注释 | X



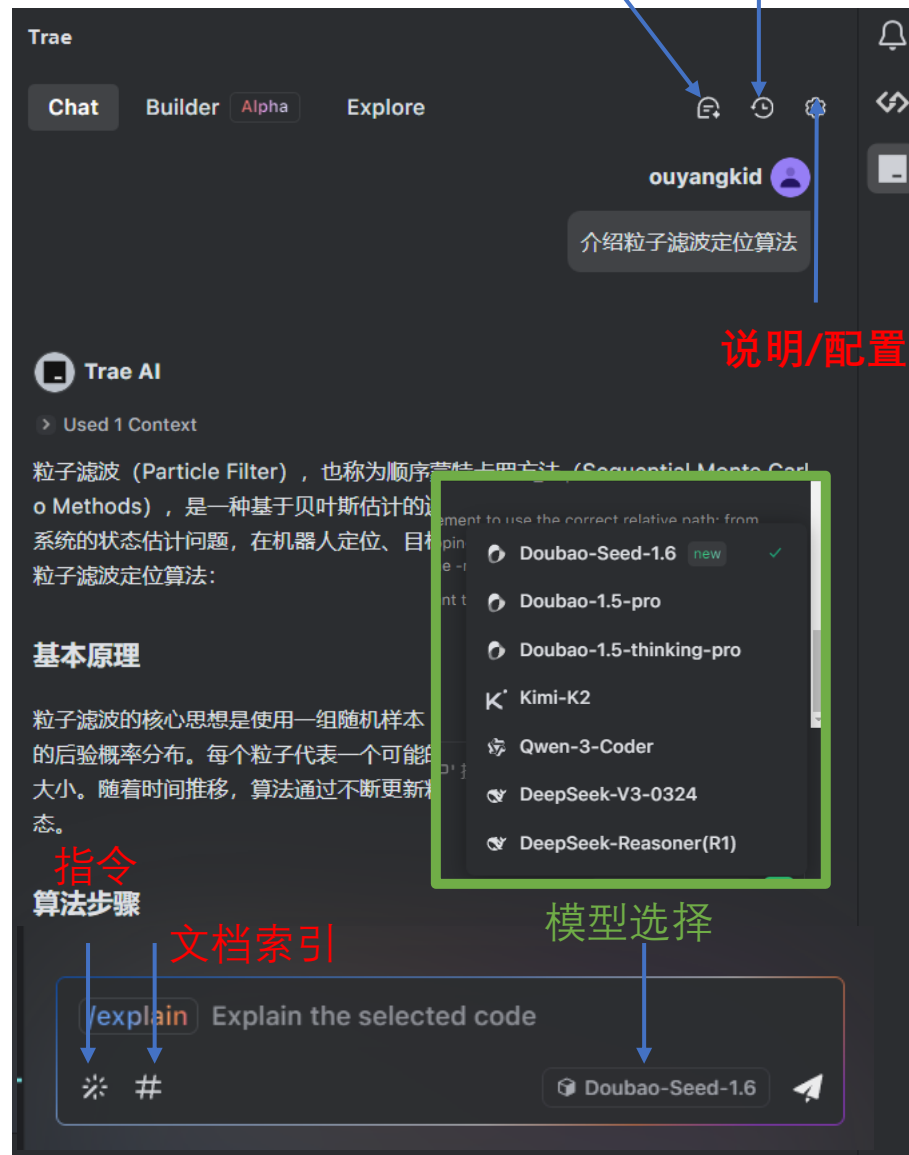
解释 | 添加注释 | X

```
def calc_input():
    v = 1.0 # [m/s]
    yaw_rate = 0.1 # [rad/s]
    u = np.array([[v, yaw_rate]]).T # 将v和yaw_rate转换为列向量
    return u # 返回输入向量u
```

另一方面，也可以利用AI助手对代码进行注释，提高算法的可理解性。

## Python环境

新对话 历史对话





# Part 5 | 软件环境准备

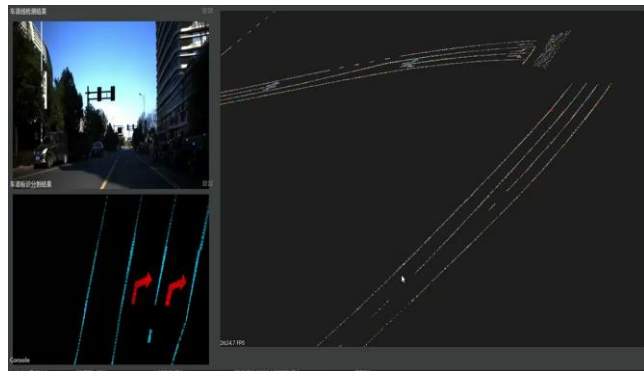
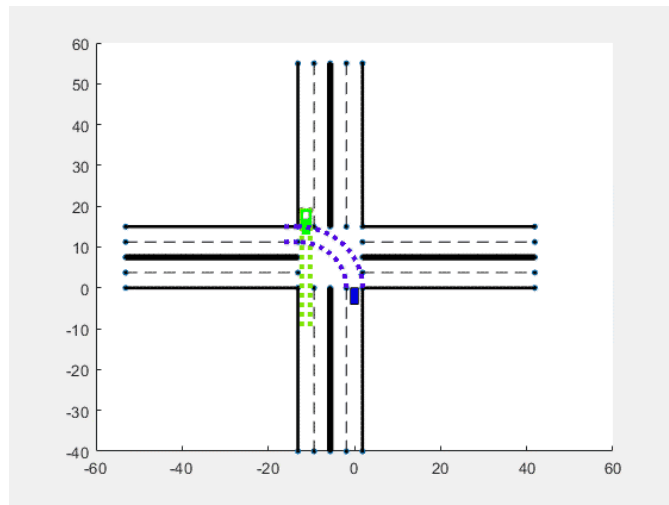
Python环境

## 示例



假设未来某天，**张三**从**学校南门**（**起点**）出发去**杭州西站**（**终点**）坐高铁，打了一辆RoboTaxi。从上车开始，无人车首先要通过GPS获取自身当前坐标（**定位**）；接着，从高精地图服务商（**建图**）获取道路级地图信息，并查找杭州西站进站口坐标（**终点**）；然后，基于**起点-终点**进行道路约束下的车道级**路径规划**，定制行车线路；最后，系统自动的控制车辆的方向盘、刹车和油门，沿着规划线路进行路径跟踪（**寻迹**）。

**不考虑**中途受其它因素导致的换道、超车（二次规划）的情况下，车辆会行驶至目标位置。



# Part 5 | 软件环境准备

### 示例demo

通过运行demo代码， 1) 验证环境安装正确性； 2) 理解课程对机器人的抽象层次； 3) 尝试使用AI插件交互学习

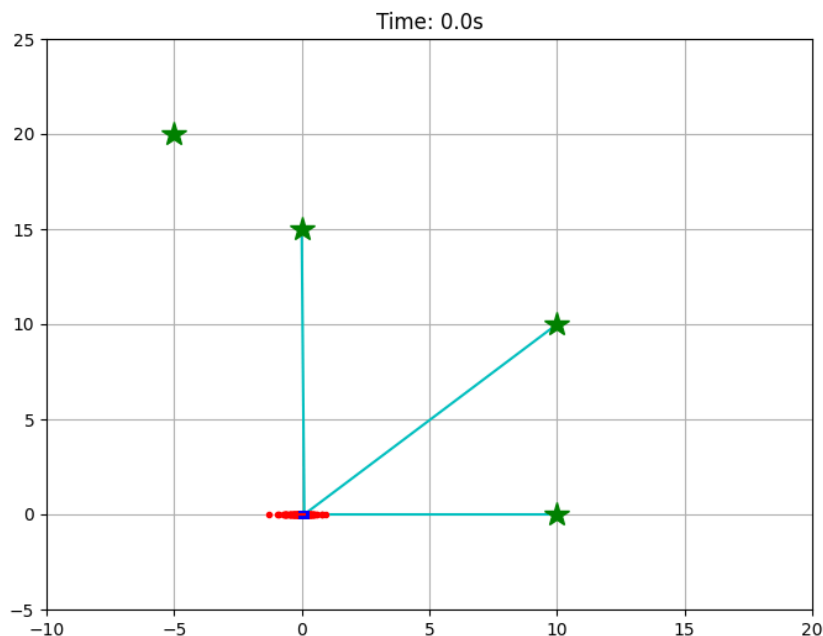
dijkstra.py

particle\_filter.py

pure\_pursuit.py

ray\_casting\_grid\_map.py

- **定位：粒子滤波算法**
- 建图：光线投影网格地图
- 路径规划：基于网格地图的 *Dijkstra*
- 寻迹：纯追踪跟踪



- 点击运行按钮，matplotlib绘制并实时更新算法结果。
- **蓝色**：GT轨迹，基本与红色轨迹重合。
- **黑色**：航位推算轨迹（存在累积误差）
- **红色**：粒子滤波算法结果（实时粒子，历史轨迹）
- **绿色五角星**：为锚点传感器（如RFID、UWB）位置，并假设机器人能够通过传感器对固定信标进行测距（即为动态刷新时当前节点与五角星的**连线**）。基础为三角定位。
- 定位过程中，本地节点有策略的选择近邻锚点。

## 粒子滤波定位

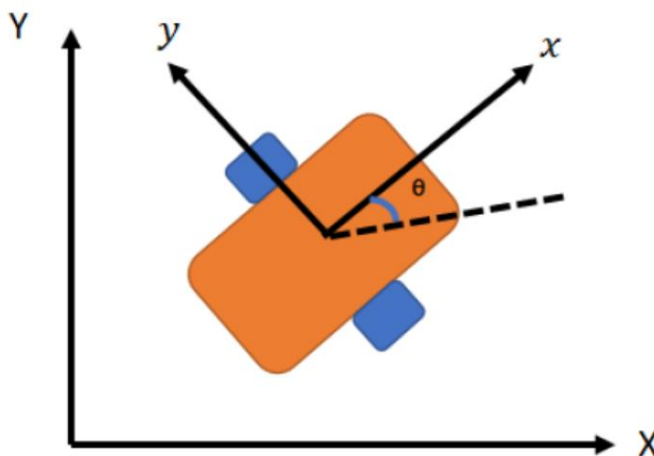
### 2D运动模型

- 2D 刚体运动模型描述了平面上物体（如机器人）的位置和姿态随时间的变化规律。
- 在平面运动中，刚体的状态由位置  $(x, y)$  和航向角Yaw  $(\theta)$  确定，构成 3 维状态向量：

$$\text{state} = [x, y, \theta]^T$$

- 在 2D 刚体运动模型中加入速度状态 $V$ （即扩展为 4 维状态向量）可以更准确地描述系统动态特性，特别是在处理非连续控制输入或需要预测未来轨迹的场景中。

$$\text{state} = [x, y, \theta, v]^T$$



时序变化关系：

$$x(i+1) = x(i) + v \cdot \Delta t \cdot \cos(\vartheta(i))$$

$$y(i+1) = y(i) + v \cdot \Delta t \cdot \sin(\vartheta(i))$$

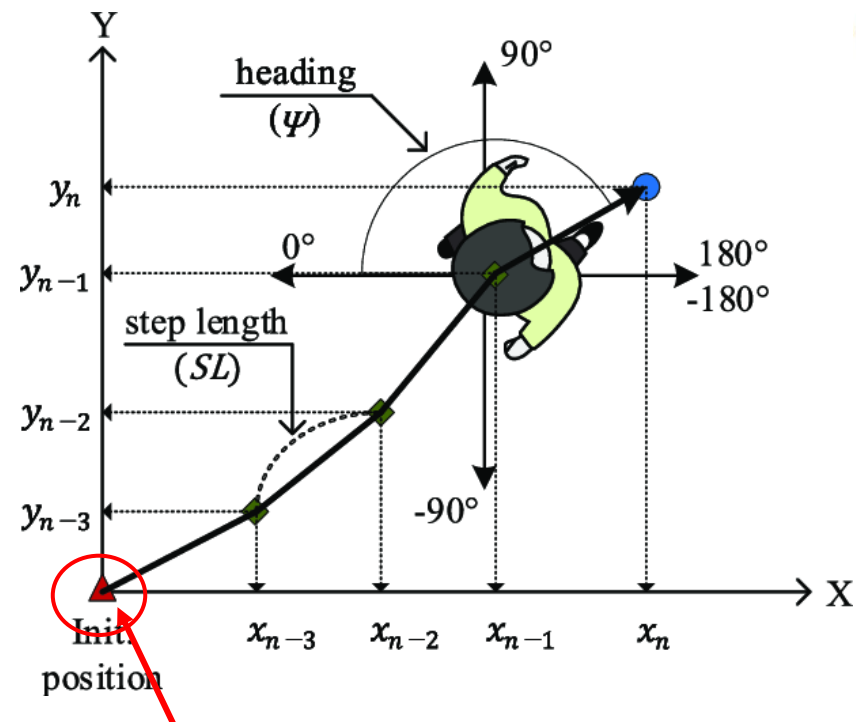
$$\vartheta(i+1) = \vartheta(i) + r \cdot \Delta t$$

# Part 5 | 软件环境准备

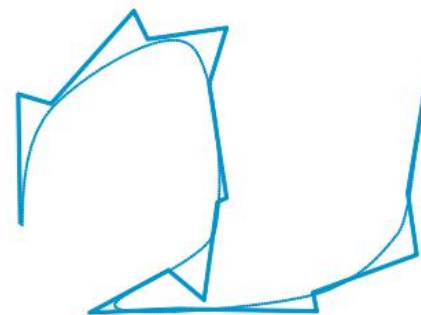
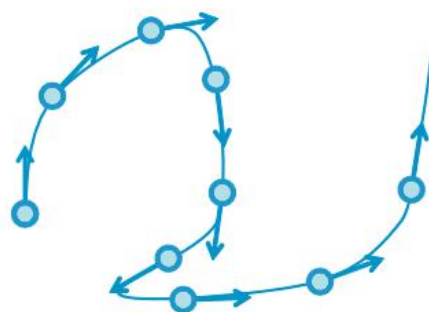
Python环境

## 粒子滤波定位

### 航迹推算 Dead Reckoning



1<sup>st</sup> Order Model



通过控制矩阵来描述，DT是采样时间， $X_t=[x,y,yaw,v]$ 是机器人状态向量[2,0]是朝向

若初始位置为  $(x_0, y_0)$ ，某段时间内移动距离为  $d_i$ 、航向角为  $\vartheta_i$ ，则新位置  $(x_n, y_n)$  为：

$$X_n = X_0 + \sum_{k=0}^n d_i \cos \theta_i$$
$$Y_n = Y_0 + \sum_{k=0}^n d_i \sin \theta_i$$

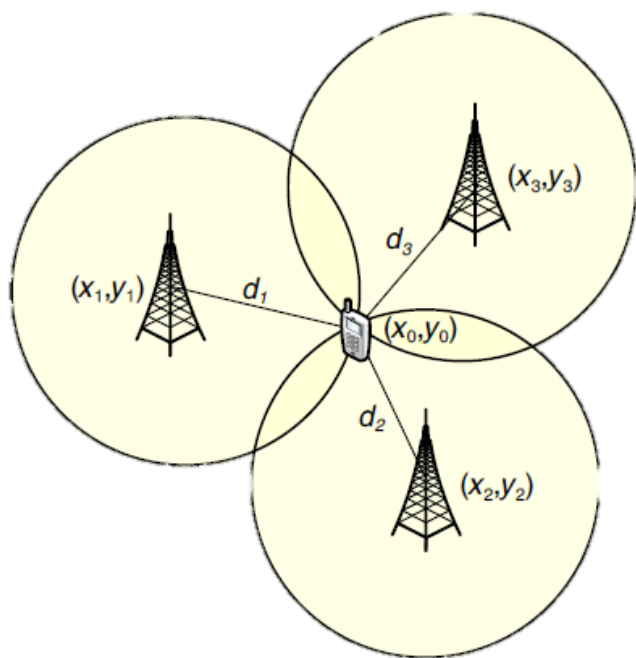


```
B = np.array([[DT * math.cos(x[2, 0]), 0],
              [DT * math.sin(x[2, 0]), 0],
              [0.0, DT],
              [1.0, 0.0]])
x = F.dot(x) + B.dot(u)
```

- 利用现在**物体位置及速度**推定未来位置方向的航海技术，容易受噪声误差影响，形成累积误差。
- DR一般依赖自身传感器进行测角（磁力计、陀螺仪）和测速（轮速计、加速度计），不依赖外部信息。

### 粒子滤波定位

#### 三角定位



- 已知三角形的三个顶点中两个顶点的位置，以及目标与这两个顶点的距离或角度关系，可通过几何计算确定第三个顶点（**目标**）的位置。
- 平面场景：至少需要 2 个参考点 + 目标到两点的距离（或角度）。
- 三维场景：至少需要 3 个参考点（形成立体三角），才能确定目标的三维坐标（如 GPS 定位）。

#### 基于距离的三角定位（Range-based）

通过测量目标到多个参考点的**距离**，利用“圆（或球）的交点”确定位置。

在时间同步的情况下，距离一般通过无线电信号收发的时间（信号内容带有时间戳） $t$ ，即光速一定 $v$ ，距离 $d=vt$ 可得。

假设：机器人能够通过传感器获取自身到锚点的**距离**，从而利用三角定位估计自身位置。因此，设定若干固定点坐标，假设短时间内它们的**绝对位置**静止。

```
rf_id = np.array([[10.0, 0.0],  
                  [10.0, 10.0],  
                  [0.0, 15.0],  
                  [-5.0, 20.0]])
```

```
for i in range(len(rf_id[:, 0])):  
    dx = x_true[0, 0] - rf_id[i, 0]  
    dy = x_true[1, 0] - rf_id[i, 1]  
    d = math.hypot(dx, dy)
```





核心思想是通过大量“粒子”（模拟目标可能位置的样本）来近似目标的概率分布，进而根据观测数据不断更新粒子权重，最终筛选出最可能的目标位置。

依赖： 机器人运动模型、观测结果（即传感器测量信息）

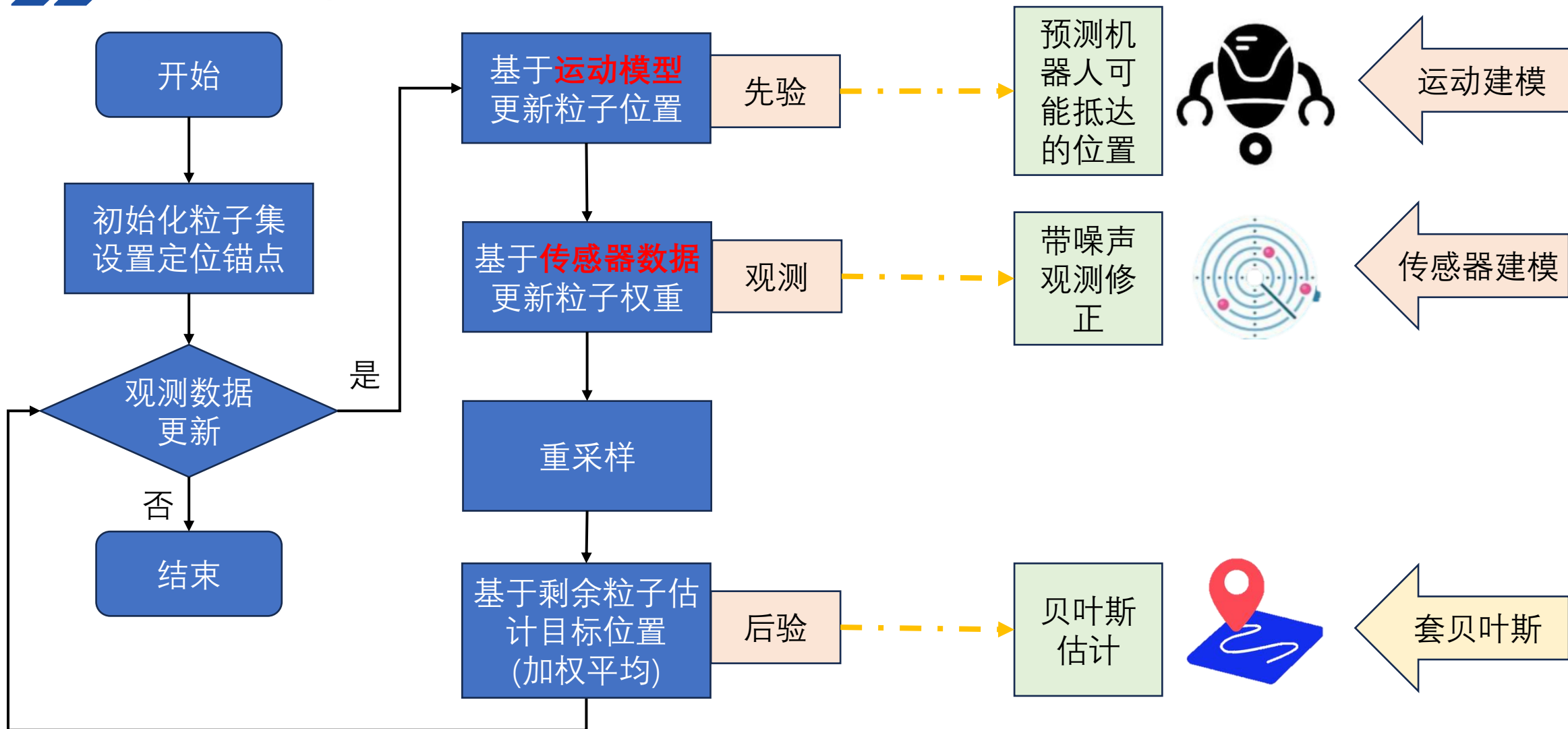
流程：

- **预测阶段**：根据系统运动模型（如目标的速度、加速度），对粒子进行“移动”，模拟目标可能的位置变化（引入随机噪声，体现运动不确定性）。
- **更新阶段**：结合传感器观测数据（如**距离**），计算每个粒子与观测数据的匹配度（似然概率），并以此更新粒子权重（距离越近，匹配度越高，权重越大）。
- **重采样阶段**：为避免权重过低的粒子浪费计算资源，保留高权重粒子并复制，低权重粒子被淘汰，使粒子集合始终聚焦于高概率区域。

# Part 5 | 软件环境准备

Python环境

## 粒子滤波定位



# Part 5 | 软件环境准备

Python环境



## 粒子滤波定位



运动建模

质点运动模型  
Dead Reckoning

机器人状态-数据结构

```
# State Vector [x y yaw v]
x_est = np.zeros((4, 1))
x_true = np.zeros((4, 1))
```

恒定速度和角速度的圆周运动模拟

```
v = 1.0 # [m/s]
yaw_rate = 0.1 # [rad/s]
u = np.array([[v, yaw_rate]]).T
```

```
def motion_model(x, u):
    # 状态转移矩阵F (4x4) : [x, y, yaw, v]的状态更新
    F = np.array([[1.0, 0, 0, 0],
                  [0, 1.0, 0, 0],
                  [0, 0, 1.0, 0],
                  [0, 0, 0, 0]])

    # 控制矩阵B (4x2) : 将控制输入转换为状态变化
    B = np.array([[DT * math.cos(x[2, 0]), 0], # x方向位移 = 速度*时间*cos(航向角)
                  [DT * math.sin(x[2, 0]), 0], # y方向位移 = 速度*时间*sin(航向角)
                  [0.0, DT], # 航向角变化 = 角速度*时间
                  [1.0, 0.0]]) # 速度保持不变

    x = F.dot(x) + B.dot(u) # 状态更新:  $x_{new} = F \cdot x + B \cdot u$ 
    return x
```

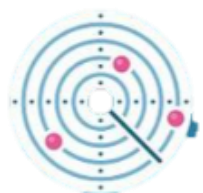
原速度 + 当前时间片增量

# Part 5 | 软件环境准备

Python环境



## 粒子滤波定位



传感器建模

观测模型

观测模型

DR运动模型

计算锚点距离

带噪观测信息

真实运动状态

感知噪声

控制指令

DR运动模型

带噪控制指令

带噪运动状态

运动噪声

```
def observation(x_true, xd, u, rf_id):
```

```
    x_true = motion_model(x_true, u) # 真实状态更新
```

```
    z = np.zeros((0, 3)) # 观测数据: [距离, 地标x, 地标y]
```

```
    for i in range(len(rf_id[:, 0])):
```

```
        # 计算真实距离
```

```
        dx = x_true[0, 0] - rf_id[i, 0]
```

```
        dy = x_true[1, 0] - rf_id[i, 1]
```

```
        d = math.hypot(dx, dy) # 欧氏距离
```

```
        if d <= MAX_RANGE: # 仅保留有效范围内的观测
```

```
            dn = d + np.random.randn() * Q_sim[0, 0] ** 0.5 # 加入观测噪声
```

```
            zi = np.array([[dn, rf_id[i, 0], rf_id[i, 1]]])
```

```
            z = np.vstack((z, zi)) # 堆叠观测数据
```

```
    # 控制输入加入噪声 (模拟实际控制误差)
```

```
    ud1 = u[0, 0] + np.random.randn() * R_sim[0, 0] ** 0.5
```

```
    ud2 = u[1, 0] + np.random.randn() * R_sim[1, 1] ** 0.5
```

```
    ud = np.array([[ud1, ud2]]).T
```

```
    xd = motion_model(xd, ud) # 航迹推演 (Dead Reckoning) 状态更新
```

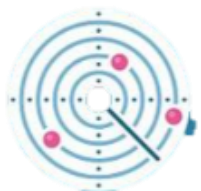
```
    return x_true, z, xd, ud
```

# Part 5 | 软件环境准备

## Python环境



### 粒子滤波定位



传感器建模

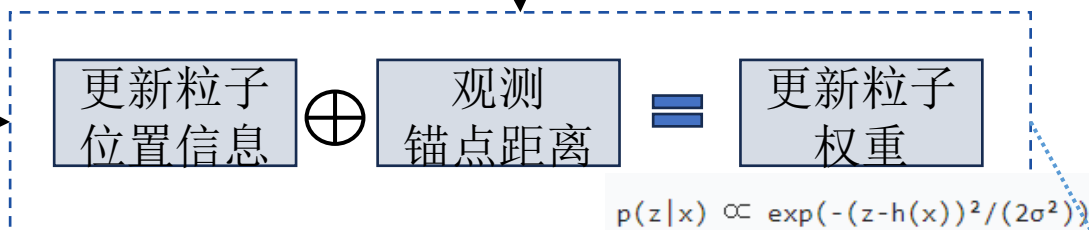
粒子滤波定位

随机预测：对每个粒子施加随机噪声，模拟系统不确定性

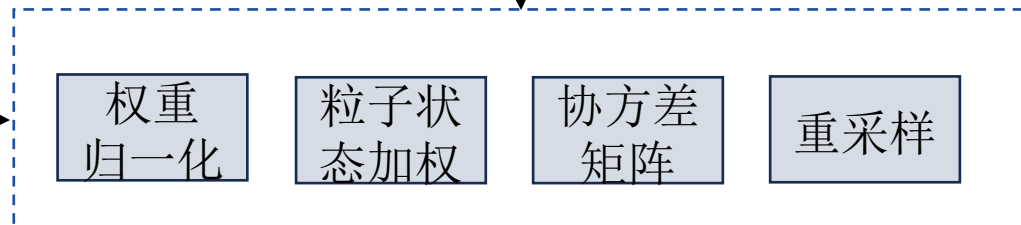
遍历所有粒子



更新粒子权重



粒子重采样



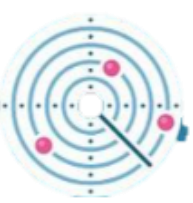
```
for ip in range(NP):  
    x = np.array([px[:, ip]]).T # 获取第ip个粒子的状态  
    w = pw[0, ip] # 获取对应权重  
  
    # 为控制输入添加噪声（模拟控制不确定性）  
    ud1 = u[0, 0] + np.random.randn() * R[0, 0] ** 0.5  
    ud2 = u[1, 0] + np.random.randn() * R[1, 1] ** 0.5  
    ud = np.array([[ud1, ud2]]).T  
  
    # 使用带噪声的控制输入更新粒子状态  
    x = motion_model(x, ud)  
  
    # 更新粒子存储和权重  
    px[:, ip] = x[:, 0]  
    pw[0, ip] = w  
  
for i in range(len(z[:, 0])): # 对每个观测数据  
    dx = x[0, 0] - z[i, 1] # 粒子位置与地标x的差值  
    dy = x[1, 0] - z[i, 2] # 粒子位置与地标y的差值  
    pre_z = math.hypot(dx, dy) # 粒子预测的与地标的距离  
    dz = pre_z - z[i, 0] # 预测距离与实际观测距离的残差  
  
    # 基于残差计算高斯似然，更新粒子权重  
    w = w * gauss_likelihood(dz, math.sqrt(Q[0, 0]))
```



# Part 5 | 软件环境准备

Python环境

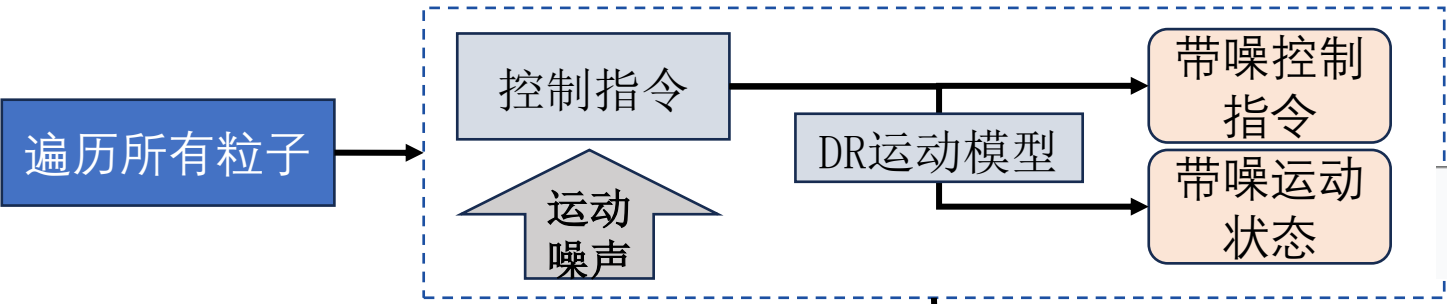
## 粒子滤波定位



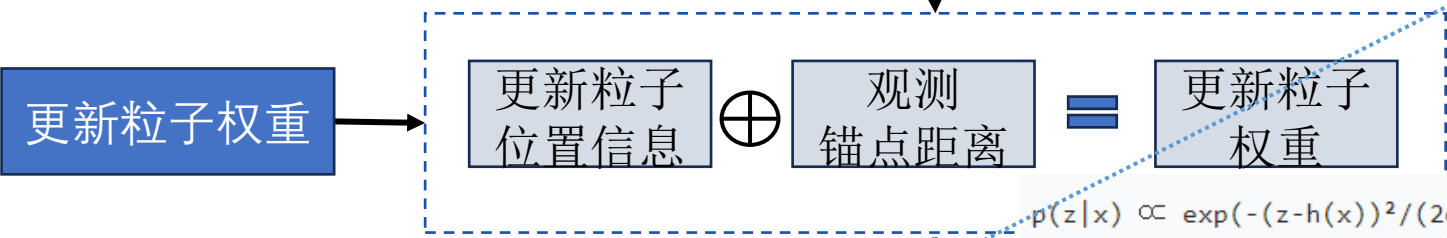
传感器建模

粒子滤波定位

随机预测：对每个粒子施加随机噪声，模拟系统不确定性

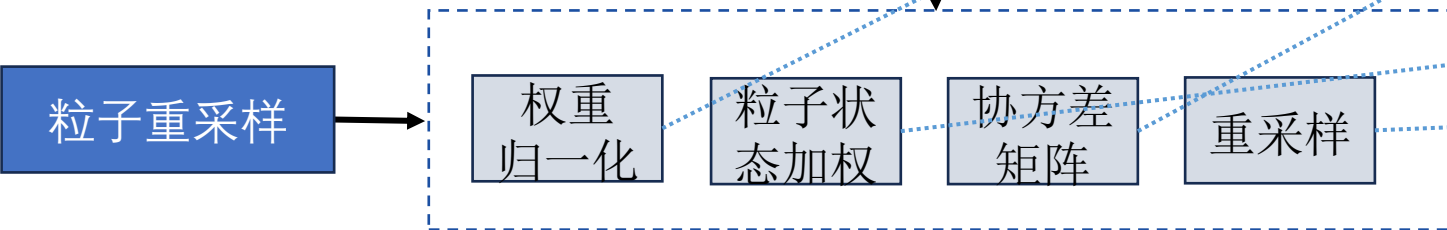


```
pw = pw / pw.sum() # 确保所有粒子权重之和为1
```



```
x_est = px.dot(pw.T) # 加权平均：粒子状态的加权和
p_est = calc_covariance(x_est, px, pw) # 计算估计的协方差矩阵
```

权重 $p_w < 1$ , 分母为 $p_w$ 的平方和  $N_{\text{eff}} = \frac{1}{\sum_{i=1}^{NP} w_i^2}$



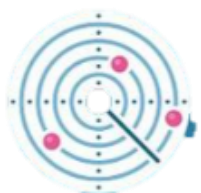
```
N_eff = 1.0 / (pw.dot(pw.T))[0, 0] # 有效粒子数
if N_eff < NTh: # 若有效粒子数低于阈值（通常为NP/2）
    px, pw = re_sampling(px, pw) # 执行重采样
```

# Part 5 | 软件环境准备

Python环境



## 粒子滤波定位



传感器建模

噪声

```
Q = np.diag([0.2]) ** 2 # 对粒子重采样时的高斯似然, 对角阵  $0.2^2$   
R = np.diag([2.0, np.deg2rad(40.0)]) ** 2 # 噪声加在 粒子采样时的 控制输入 上  
  
# Simulation parameter  
Q_sim = np.diag([0.2]) ** 2 # 感知噪声 噪声加在到锚点的距离估计上  
R_sim = np.diag([1.0, np.deg2rad(30.0)]) ** 2 # 噪声加在对机器人的 控制输入 上
```

仿真模拟  
运动过程

锚点定位

定位噪声  
 $Q_{sim}$

运动控制

控制噪声  
 $R_{sim}$

粒子滤波  
定位过程

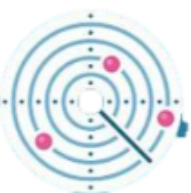
粒子运动

控制噪声  
 $R$

权重更新

$Q$ 观测不确定  
定性

### 粒子滤波定位



传感器建模

噪声

**实验要求：**调整定位锚点位置（改变分布），模拟不同移动轨迹，如双曲线、椭圆、S形等机器人运动轨迹；并分析粒子采样策略、噪声分布对算法的影响。

| 矩阵    | 类型   | 作用对象            | 物理意义                                         | 在代码中的使用                                                  |
|-------|------|-----------------|----------------------------------------------|----------------------------------------------------------|
| R     | 控制噪声 | 运动模型（控制输入 $u$ ） | 滤波算法假设的 <b>控制执行误差</b> （如电机精度不足导致的速度 / 角速度偏差） | 在 <code>pf_localization()</code> 中为每个粒子的控制输入添加噪声         |
| R_sim | 控制噪声 | 运动模型（控制输入 $u$ ） | 仿真环境中 <b>真实的控制执行误差</b>                       | 在 <code>observation()</code> 中生成带噪声的控制输入 <code>ud</code> |
| Q     | 观测噪声 | 观测模型（距离测量）      | 滤波算法假设的 <b>距离观测误差</b> （如 TOF 传感器的测量噪声）       | 在 <code>gauss_likelihood()</code> 中计算粒子权重时作为观测噪声方差       |
| Q_sim | 观测噪声 | 观测模型（距离测量）      | 仿真环境中 <b>真实的距离观测误差</b>                       | 在 <code>observation()</code> 中生成带噪声的锚点观测                 |

### 粒子滤波定位算法总结：

目标运动（如位置、速度）和传感器观测（如距离、角度测量）都存在噪声（随机性），导致无法通过确定性模型（如航迹推演）精确预测位置。

粒子滤波通过用**大量粒子（分身）模拟这些随机性**，每个粒子代表一种“可能的运动状态”（包含位置、速度等信息）。

粒子集的分布（数量、位置）理论上需要覆盖所有“合理的可能状态”。当粒子数趋于无穷时，粒子集的分布会逼近真实的**后验概率分布**（贝叶斯滤波的理想解）。

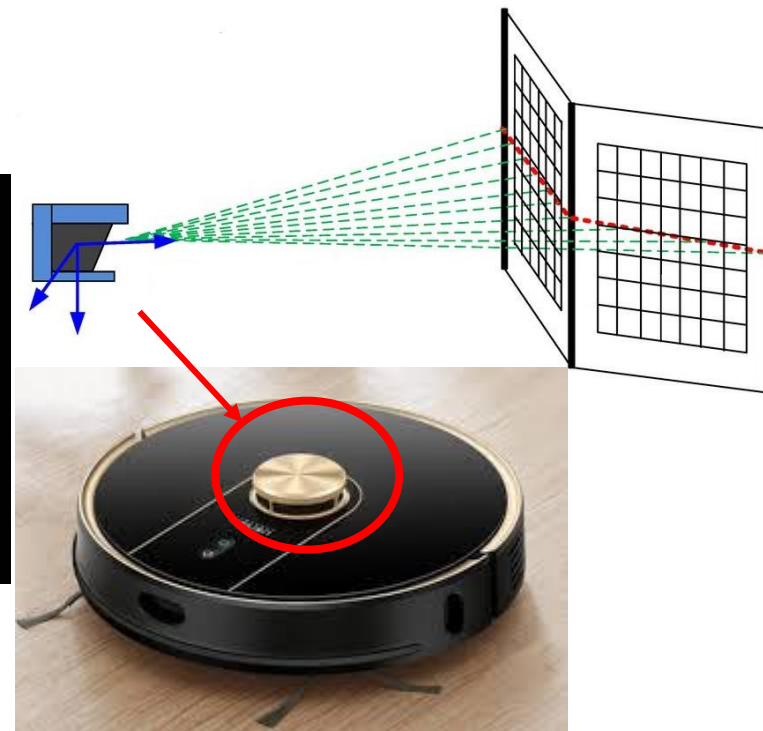
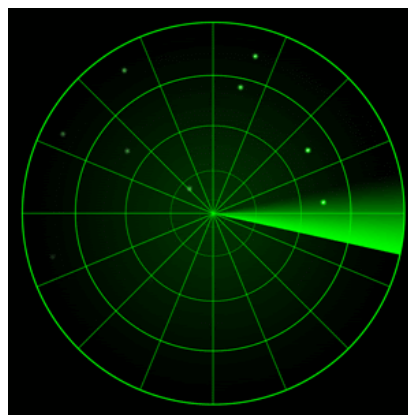
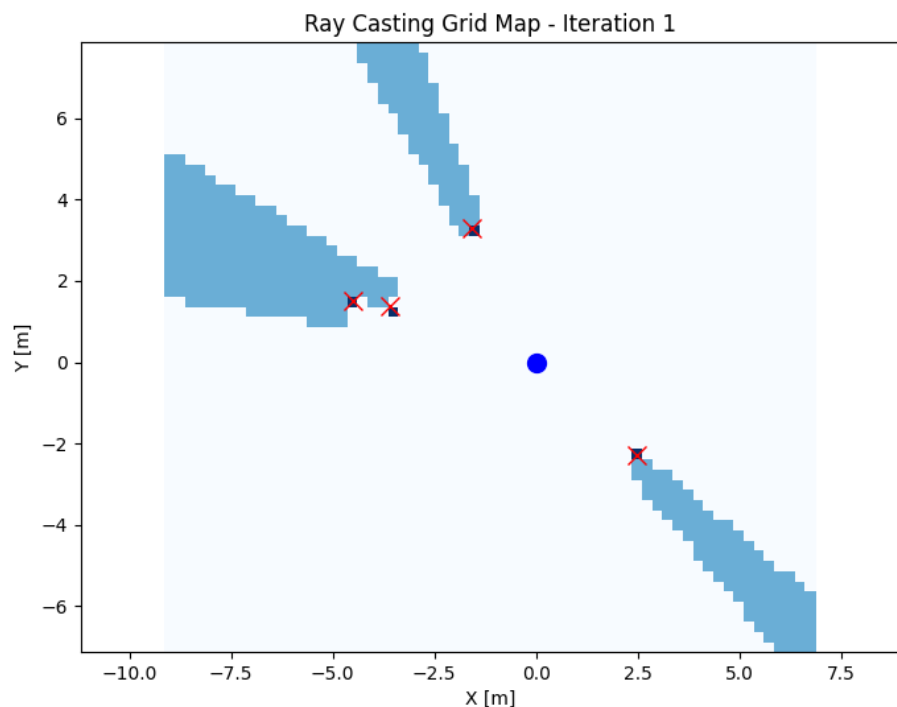
每次观测通过**权重计算**（粒子与观测值的匹配度）筛选出“更可能符合真实状态”的粒子（权重高）；权重高的粒子被复制保留，权重低的被淘汰，粒子集逐渐聚焦到真实状态附近（后验概率收敛）。

# Part 5 | 软件环境准备

Python环境

## 光线投影网格地图

- 定位：粒子滤波算法
- **建图：光线投影网格地图**
- 路径规划：基于网格地图的 *Dijkstra*
- 寻迹：纯追踪跟踪



2D激光雷达：基于红外非可见光波段的激光ToF (Time of Fly) 收发时间测距( $D=C*T/2$ ，即T时间内光走过距离的一半)。

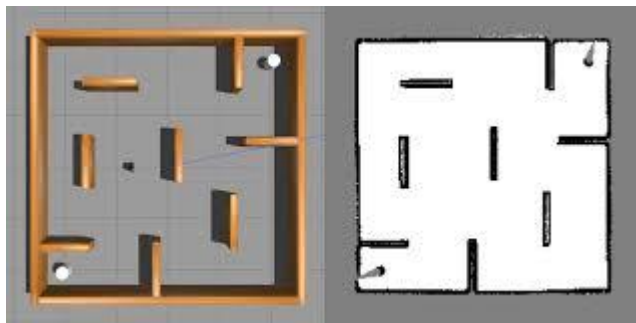
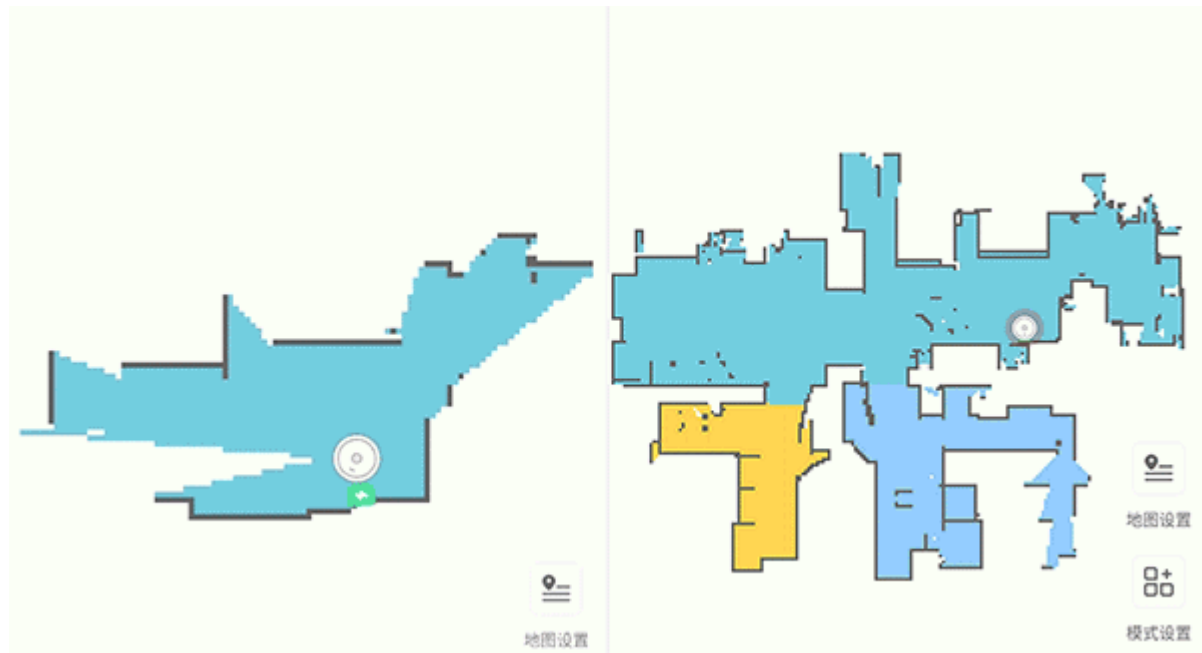
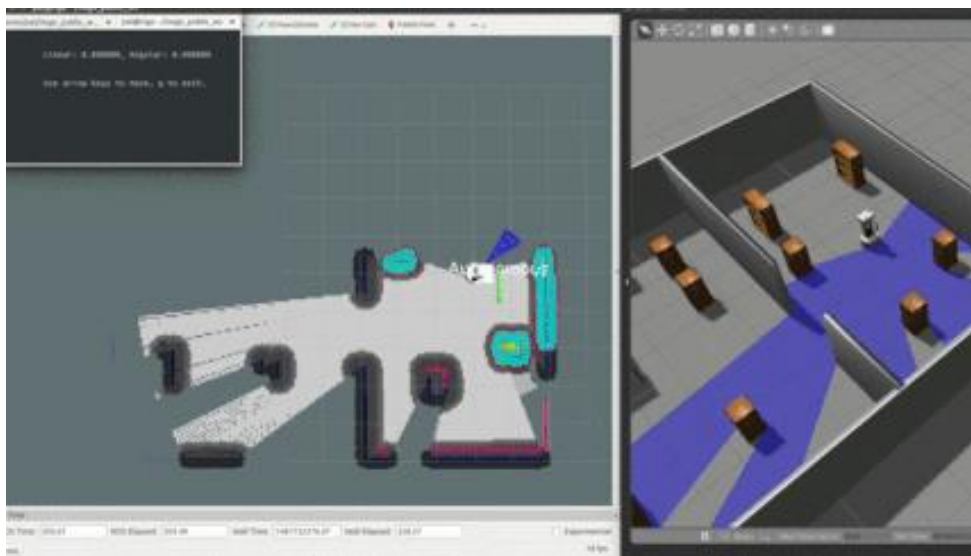
- 点击运行按钮，matplotlib绘制并实时更新算法结果。
- **深蓝色**：传感器（机器人）当前位置。
- **红色X**：障碍物位置，占据1个单位
- **浅蓝色**：非视距遮挡部分。
- 其余：空白区域

# Part 5 | 软件环境准备

Python环境

## 光线投影网格地图

- 定位：粒子滤波算法
- **建图：光线投影网格地图**
- 路径规划：基于网格地图的 *Dijkstra*
- 寻迹：纯追踪跟踪



2d地图：

- 仅包含轮廓、缺少语义特征
- 无高度测量
- 分辨率和测距有限
- 数据量小、计算简单、配准速度快

将移动机器人移动过程中的扫描结果关联起来，就是SLAM



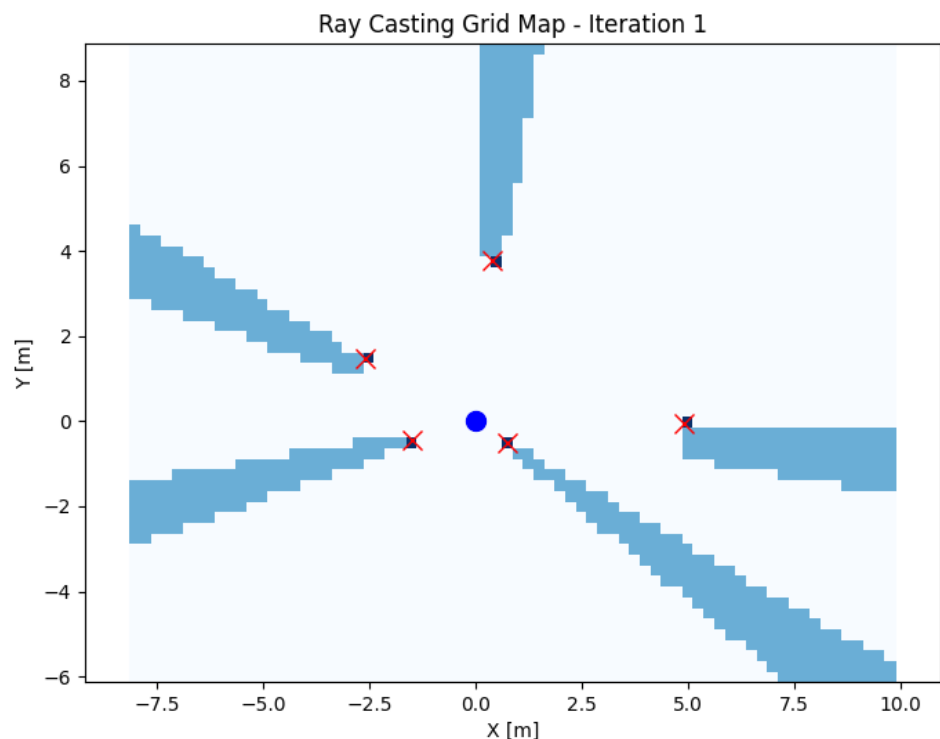
# Part 5 | 软件环境准备

Python环境



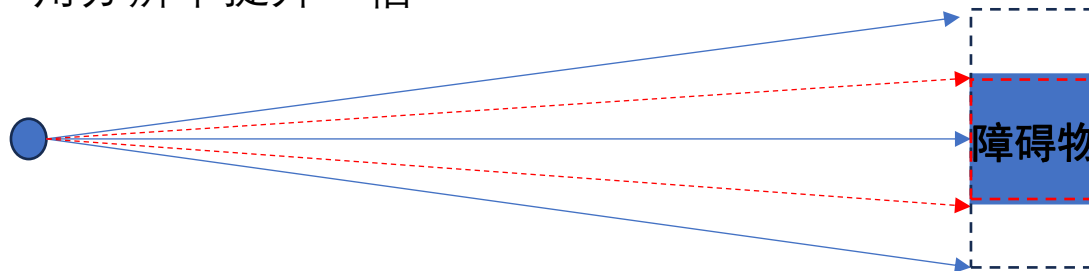
## 光线投影网格地图

- 激光雷达



逐渐增加激光雷达的角分辨率，使得对地图中点目标的遮挡测量更加精细

角分辨率提升一倍



|       | M10           | M10P  | M10 PHY |
|-------|---------------|-------|---------|
| 角度分辨率 | 0.36° / 0.72° | 0.22° | 0.36°   |
| 扫描频率  | 10 / 20Hz     | 12Hz  | 10Hz    |



早期单线雷达在测量过程中，通过电机旋转，带动激光器从点测量，提升至360度线测量。激光器的收发频率和电机旋转速度，是制约其扫描分辨率的瓶颈。

角分辨率越高，建图越精细。

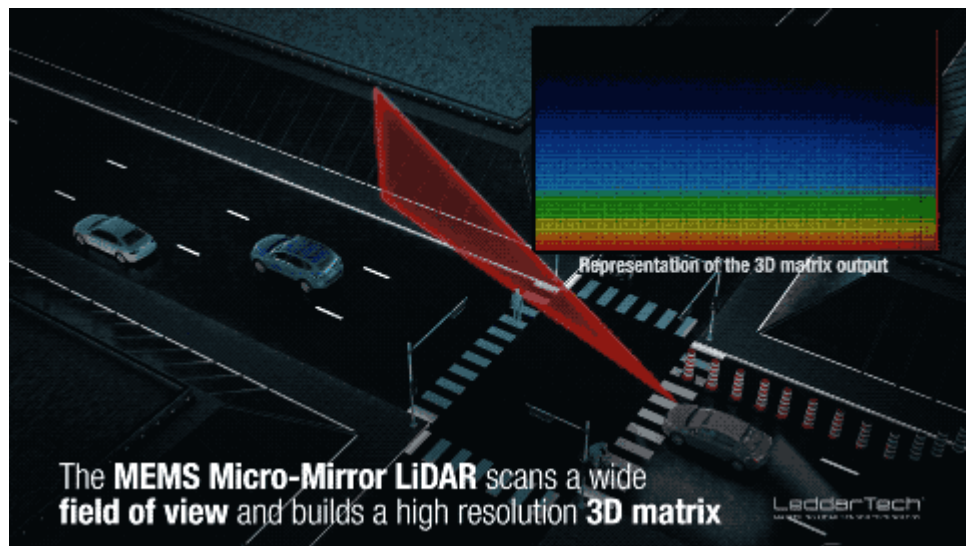
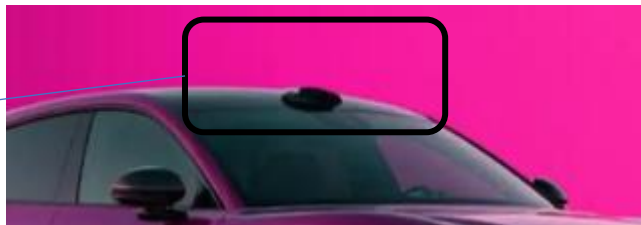
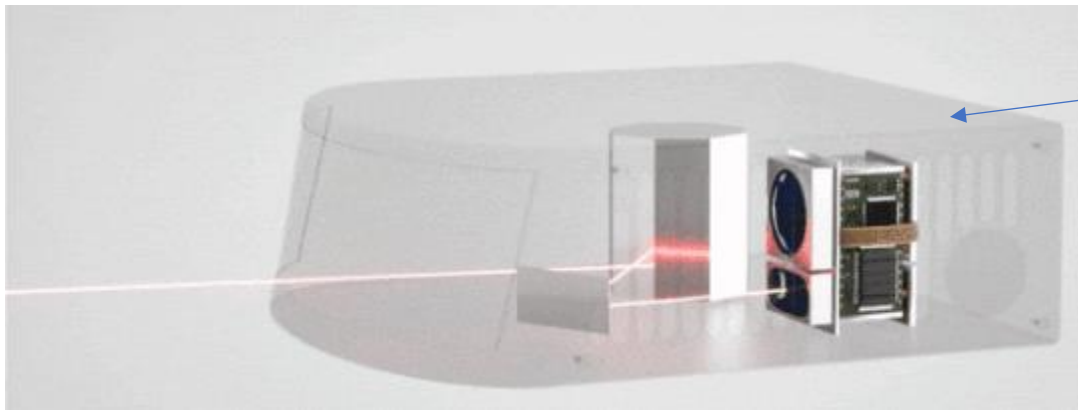
# Part 5 | 软件环境准备

Python环境



## 光线投影网格地图

- 激光雷达



当前用于辅助驾驶系统的激光雷达，基本为3D。通过内置的摆镜和转镜，提升扫描的空间自由度，进而获得更加稠密的空间点云，用于目标识别、追踪和预测。

而激光器也同样采用更高频的收发、解析和加密技术，更高的功率来提升测距距离。

相关的数据处理暂不在本课程讨论。

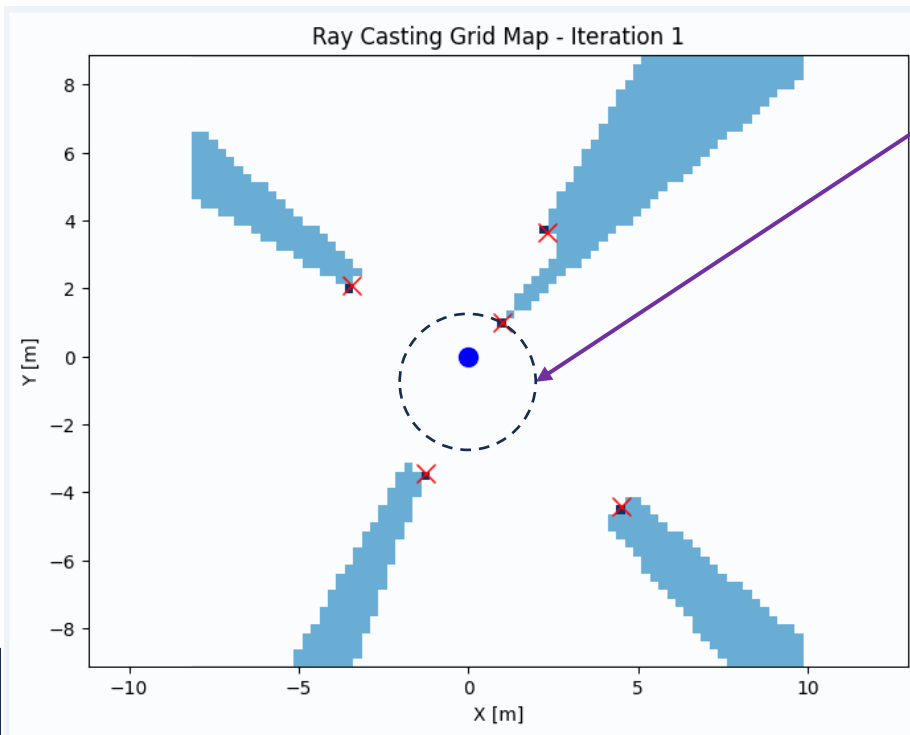
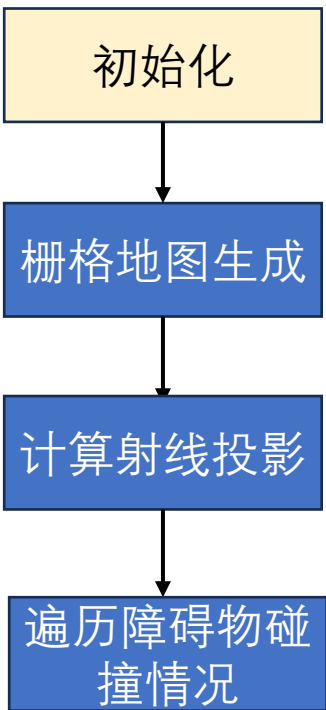
# Part 5 | 软件环境准备

Python环境



## 光线投影网格地图

坐标系： 传感器坐标系，一般以传感器自身中心为**感知空间**的原点，进而基于（本体较大时：如公交、矿车）机器人进行**外参标定**。



栅格定义：定义栅格地图的分辨率（世界建模精度），以及传感器扫描角分辨率（感知精度）

```
xyreso = 0.25 # 栅格分辨率 [m]
yawreso = np.deg2rad(5) # 角度分辨率 [rad]
```

障碍物：随机生成分布于2D空间中的若干障碍物。

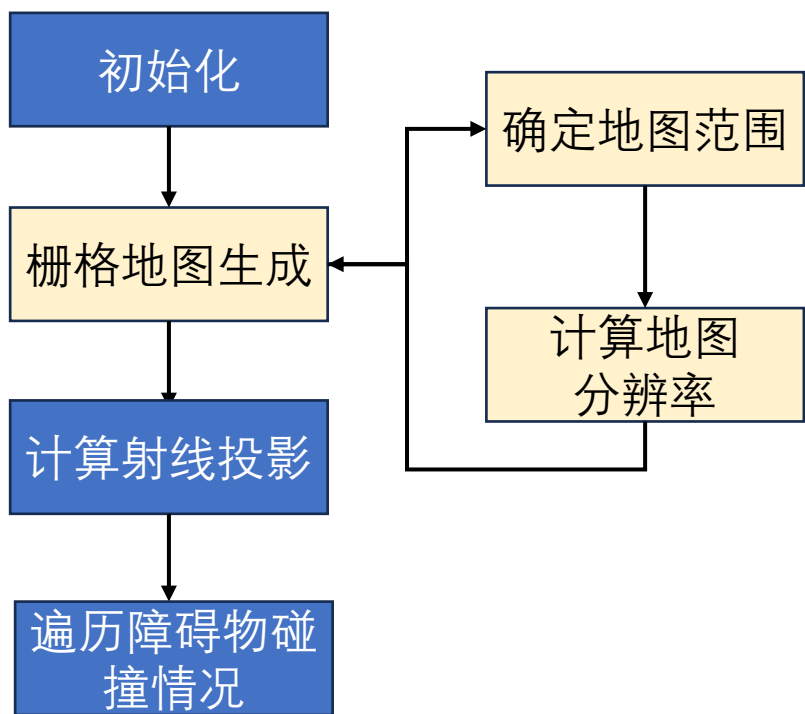
```
# 生成随机障碍物
obstacles = 5
ox = (np.random.rand(obstacles) - 0.5) * 10.0
oy = (np.random.rand(obstacles) - 0.5) * 10.0
```

# Part 5 | 软件环境准备

Python环境

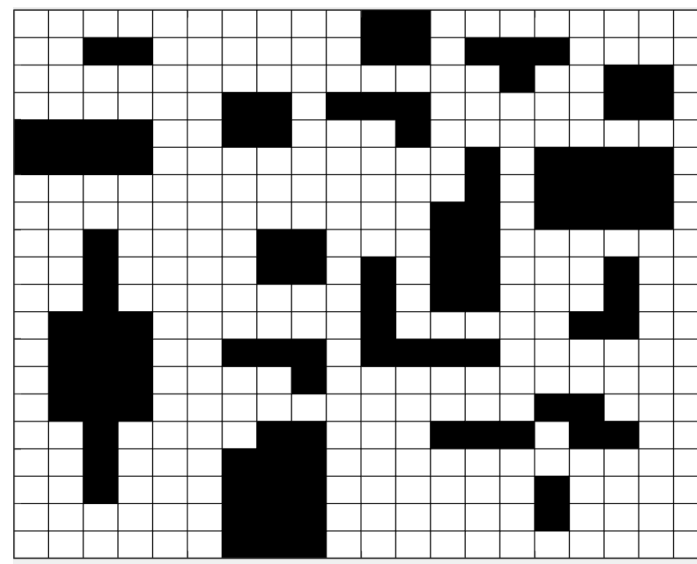


## 光线投影网格地图



基于障碍物坐标外扩

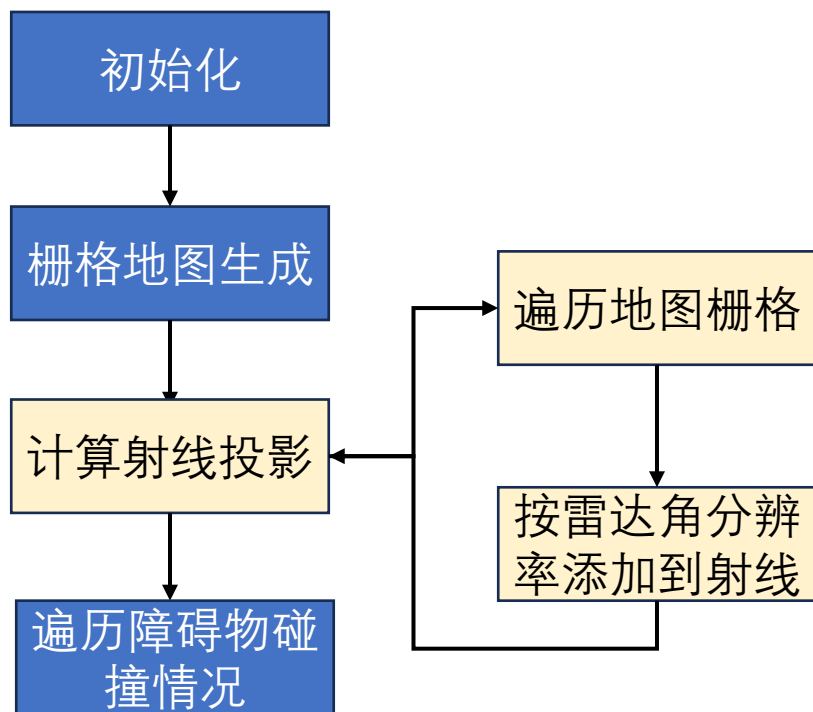
```
minx = round(min(ox) - EXTEND_AREA / 2.0)
miny = round(min(oy) - EXTEND_AREA / 2.0)
maxx = round(max(ox) + EXTEND_AREA / 2.0)
maxy = round(max(oy) + EXTEND_AREA / 2.0)
xw = int(round((maxx - minx) / xyreso))
yw = int(round((maxy - miny) / xyreso))
```



# Part 5 | 软件环境准备

Python环境

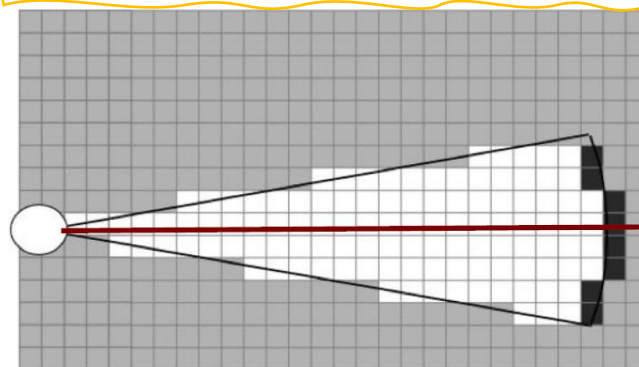
## 光线投影网格地图



不考虑障碍物遮挡的情况下，分析每条射线覆盖的地图栅格。

- 遍历每个栅格，计算其物理坐标（因为栅格化，坐标精度有损失），实际获得的是2D体素化坐标；
- 然后基于（体素坐标）计算对应角度属于哪根传感器射线。

```
for ix in range(xw):  
    for iy in range(yw):  
        px = ix * xyreso + minx  
        py = iy * xyreso + miny  
        d = math.hypot(px, py)  
        angle = atan_zero_to_twopi(py, px)  
        angleid = int(math.floor(angle / yawreso))
```



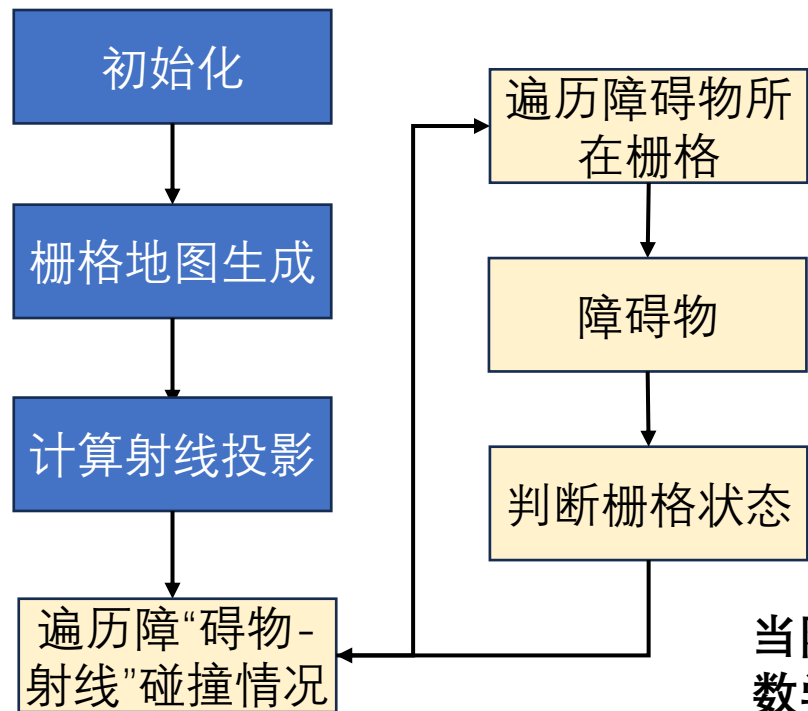


# Part 5 | 软件环境准备

Python环境



## 光线投影网格地图



到原点距离  
小于障碍物

障碍物所在栅格

到原点距离  
大于障碍物

障碍物遮挡的栅格

射线上的栅格

当障碍物是2D离散点的情况下  
数学模型：估计点到直（射）线距离  
/点是否在直线上。

```
for (x, y) in zip(ox, oy):
    d = math.hypot(x, y)
    angle = atan_zero_to_twopi(y, x)
    angleid = int(math.floor(angle / yawreso))
    gridlist = precast[angleid]

    ix = int(round((x - minx) / xyreso))
    iy = int(round((y - miny) / xyreso))

    # 将射线经过且距离大于障碍物距离的栅格设为0.5
    for grid in gridlist:
        if grid.d > d:
            pmap[grid.ix][grid.iy] = 0.5

    # 将障碍物所在栅格设为1.0
    pmap[ix][iy] = 1.0
```

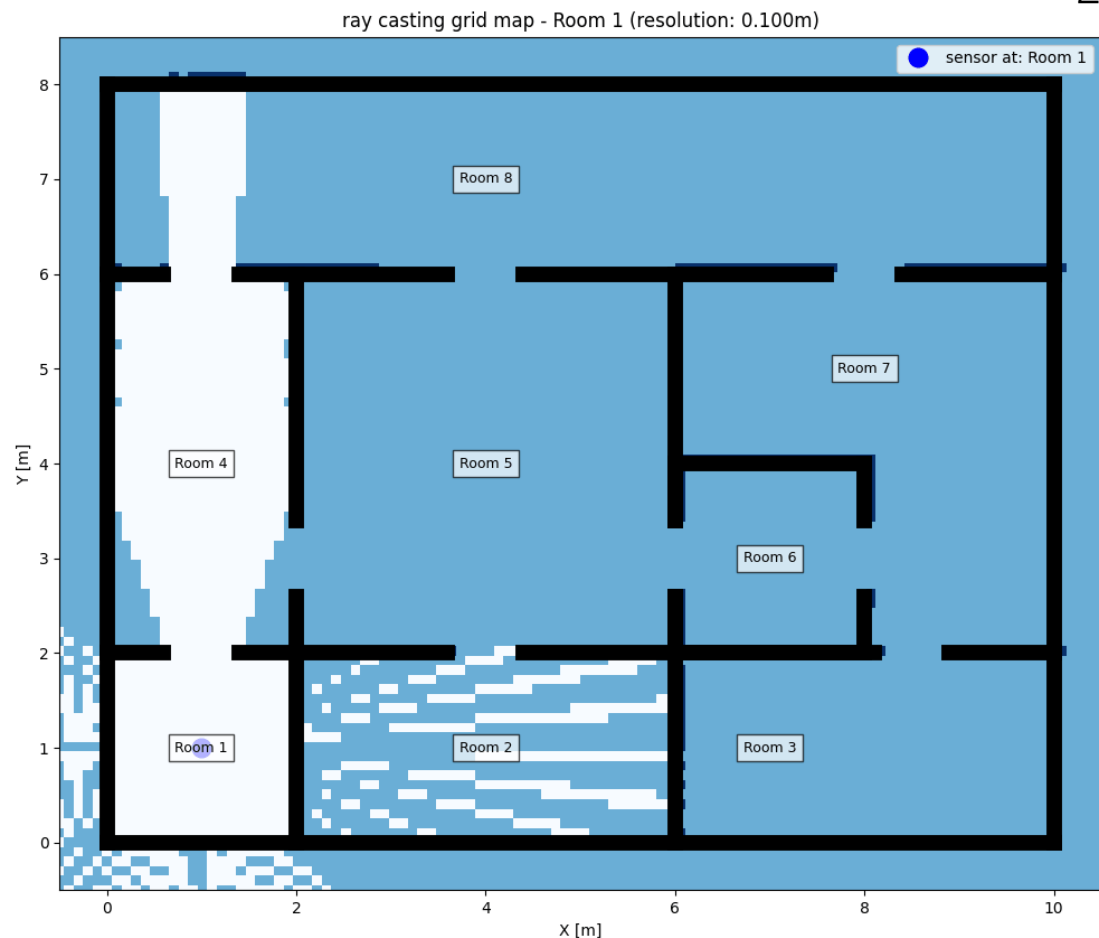
### 时空复杂度分析：

1. 最小栅格体积，影响总体的世界地图和障碍物占据数量；
2. 传感器角分辨率，影响射线数量。

# Part 5 | 软件环境准备



## 光线投影网格地图

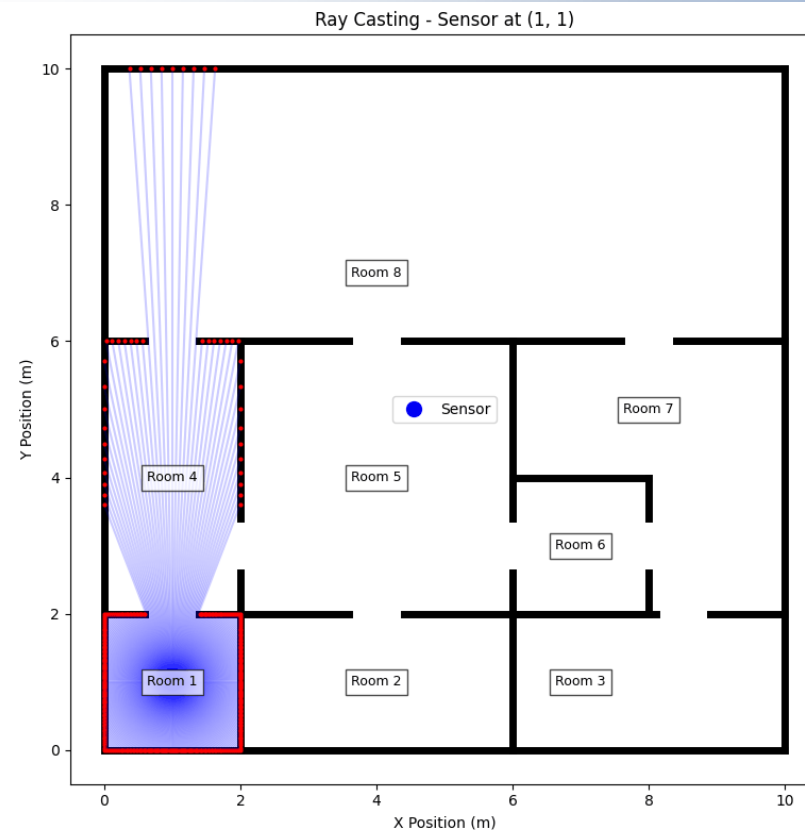


相应的，将障碍物建模为连续的墙体，即可模拟激光雷达在室内的扫描情况。

当障碍物是2D线段的情况下  
数学模型：射线到线段交点。

1. 无交点：平行
2. 交点在线段延长线上
3. 与当前线段共线

## Python环境



**实验要求：**基于自己家、实验室、教学楼或者任意网络上的房地产开发商提供的**室内平面图**，构建一个复杂室内地图/迷宫。配置传感器扫描角分辨率，并依次遍历多个不同位置，构建光线投影网格地图。

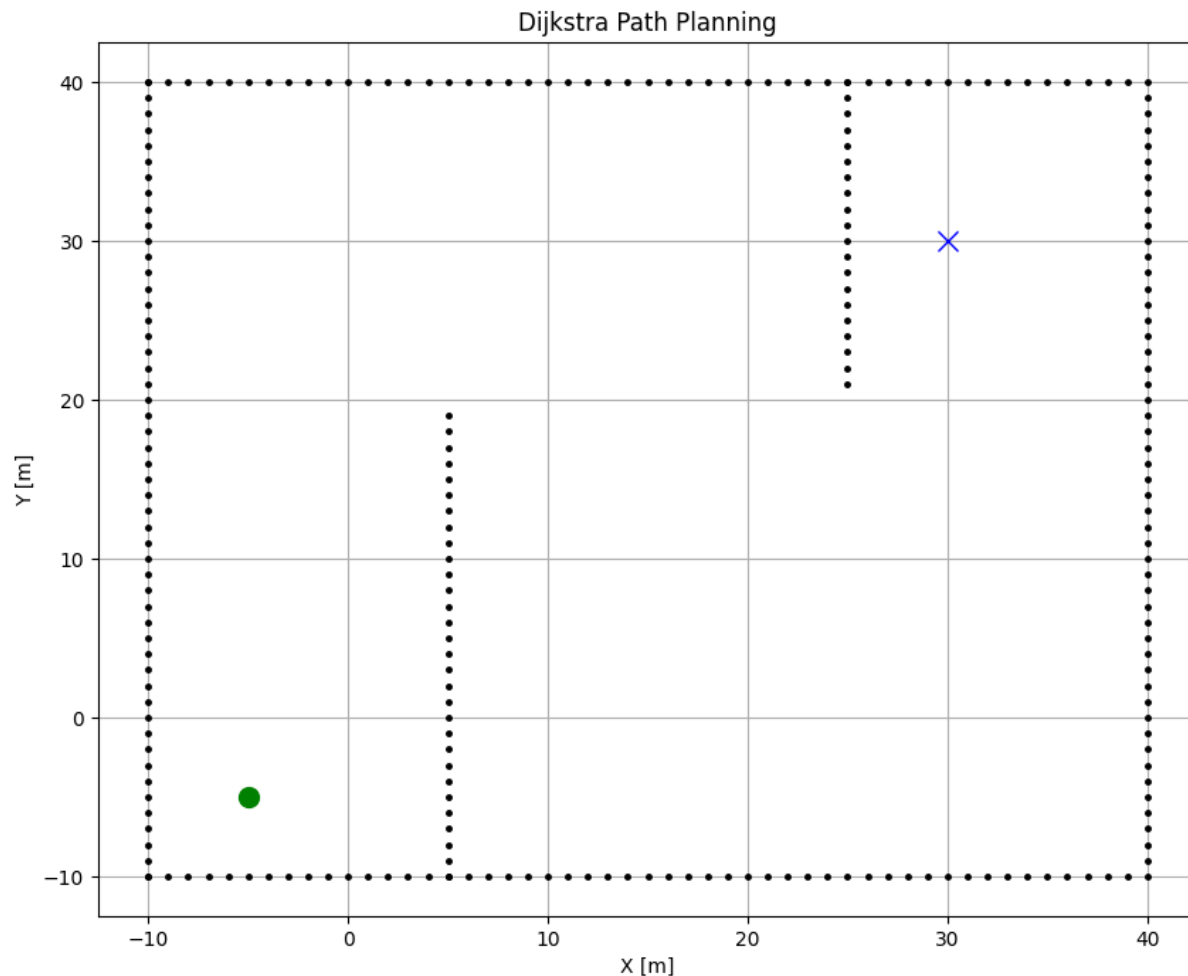
漏光现象：分析并改善。

# Part 5 | 软件环境准备

Python环境

## 路径规划

- 定位：粒子滤波算法
- 建图：光线投射网格地图
- **路径规划：基于网格地图的Dijkstra**
- 寻迹：纯追踪跟踪
- 点击运行按钮，matplotlib绘制并实时更新算法结果。
- **黑色**：墙体
- **绿色O**：起点
- **蓝色X**：终点。
- **青色X**：搜索地图
- **红色**：基于地图的最短路径规划。



# Part 5 | 软件环境准备

Python环境



## 路径规划-基于网格地图的Dijkstra

Dijkstra是经典路径规划算法，在2D情况下，可以用于迷宫路径搜索等任务。

### 基本场景构建

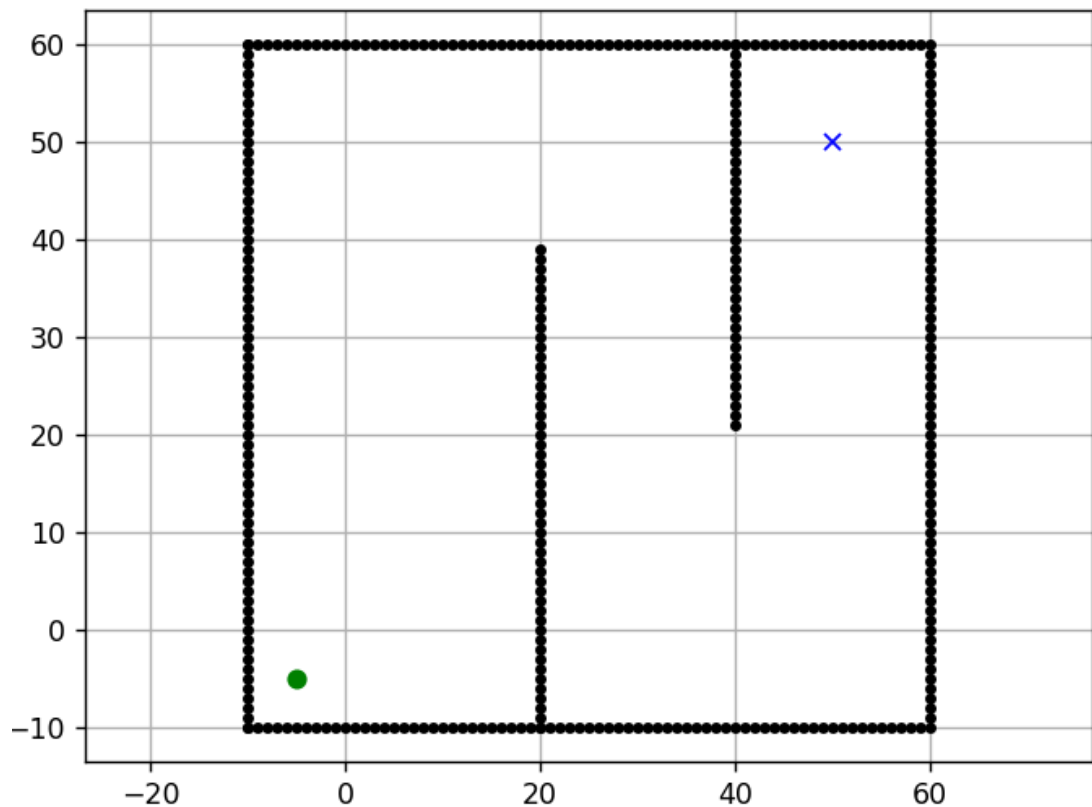
之前的算法类似，构建简易目标场景，并定义一个2D栅格地图， $x=[-10, 60]$ ,  $y=[-10, 60]$ ，分辨率为2m。  
为了使空间拓扑的复杂度增加，可以在场景中阻塞任意区块。如：line1=[(20,-10), (20,40)]，line2=[(40,20), (40,60)]。

```
# 设置障碍物位置
ox, oy = [], []

# 创建边界障碍物
for i in range(-10, 60):
    ox.append(float(i))
    oy.append(-10.0)
```

定义规划的起点(-5, -5)和终点(50,50).

```
sx, sy = -5.0, -5.0 # 起点 [m]
gx, gy = 50.0, 50.0 # 终点 [m]
grid_size = 2.0     # 网格大小 [m/cell]
robot_radius = 1.0   # 机器人半径 [m]
```



# Part 5 | 软件环境准备

Python环境



## 路径规划-基于网格地图的Dijkstra

### 基于障碍物列表构建占位地图

整个区域将由  $(70/2)^2$  个连续区块构成。

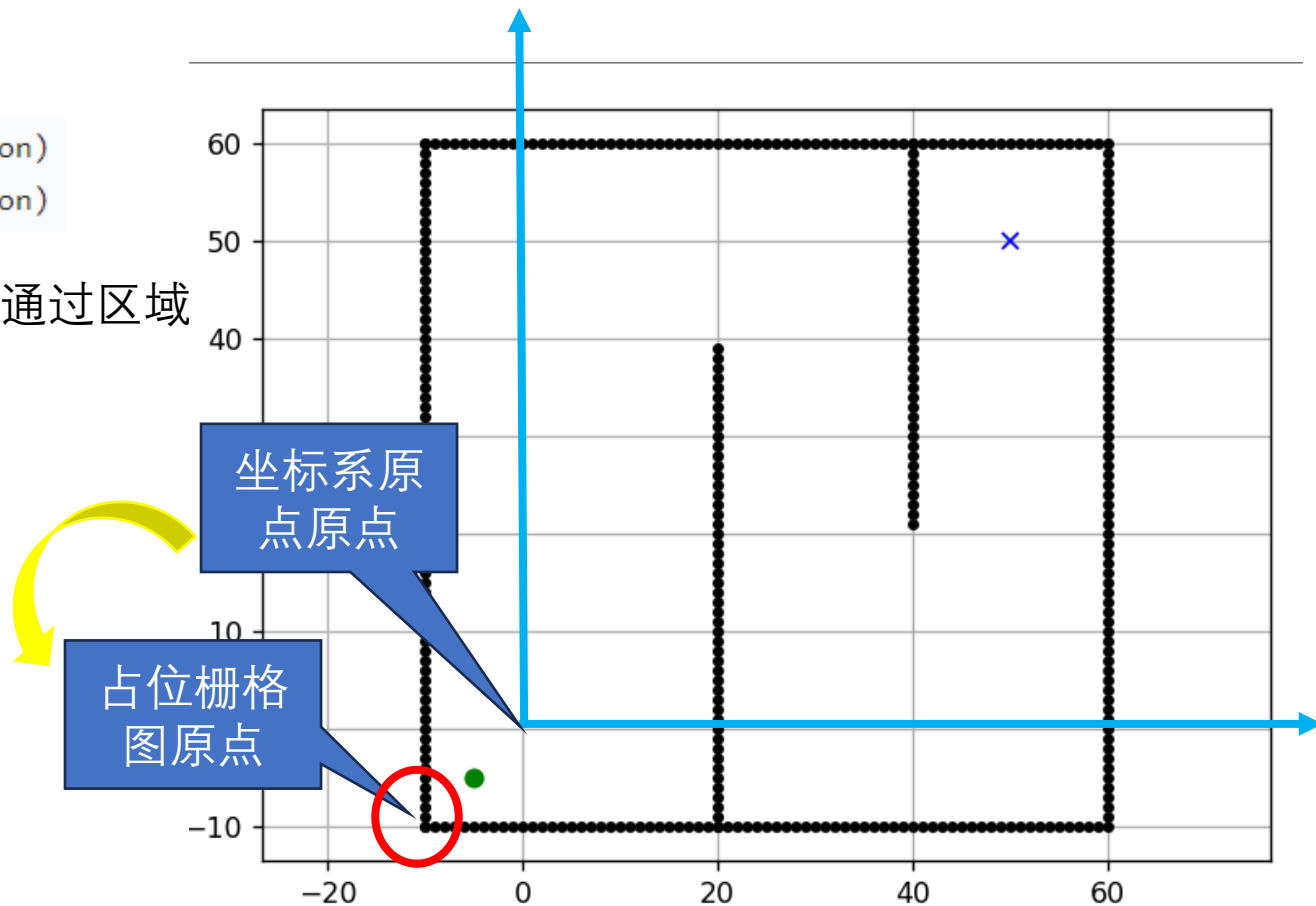
```
self.x_width = round((self.max_x - self.min_x) / self.resolution)
self.y_width = round((self.max_y - self.min_y) / self.resolution)
```

利用布尔数组构建占位地图，True=障碍物；False=可通过区域

```
self.obstacle_map = [[False for _ in range(self.y_width)]
                      for _ in range(self.x_width)]
```

遍历  
计算  
每个  
区块  
中心  
坐标  
到障  
碍物  
占据  
情况

```
for ix in range(self.x_width):
    x = self.calc_position(ix, self.min_x)
    for iy in range(self.y_width):
        y = self.calc_position(iy, self.min_y)
        for iox, ioy in zip(ox, oy):
            d = math.hypot(iox - x, ioy - y)
            if d <= self.robot_radius:
                self.obstacle_map[ix][iy] = True
                break
```





# Part 5 | 软件环境准备

Python环境



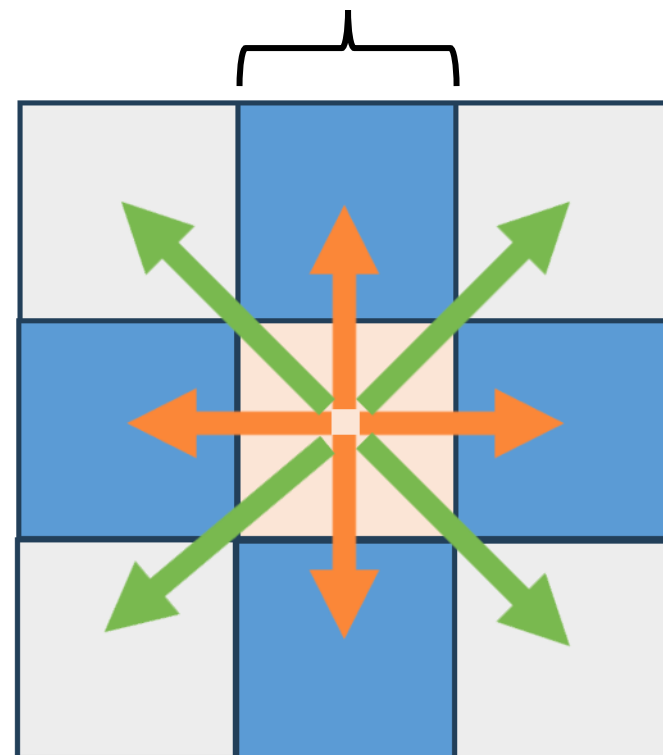
## 路径规划-基于网格地图的Dijkstra

### 简易运动模型

在栅格图中，机器人只能选择8个不同的方向，每种情形下行进的距离有两种，分别是1m和 $\sqrt{2}$ m。

注：移动距离/速度（固定量）无需与栅格区块大小相匹配，仅用于后续Dijkstra评估路径代价

```
def get_motion_model():  
    # dx, dy, cost  
    motion = [[1, 0, 1],  
              [0, 1, 1],  
              [-1, 0, 1],  
              [0, -1, 1],  
              [-1, -1, math.sqrt(2)],  
              [-1, 1, math.sqrt(2)],  
              [1, -1, math.sqrt(2)],  
              [1, 1, math.sqrt(2)]]  
  
    return motion
```



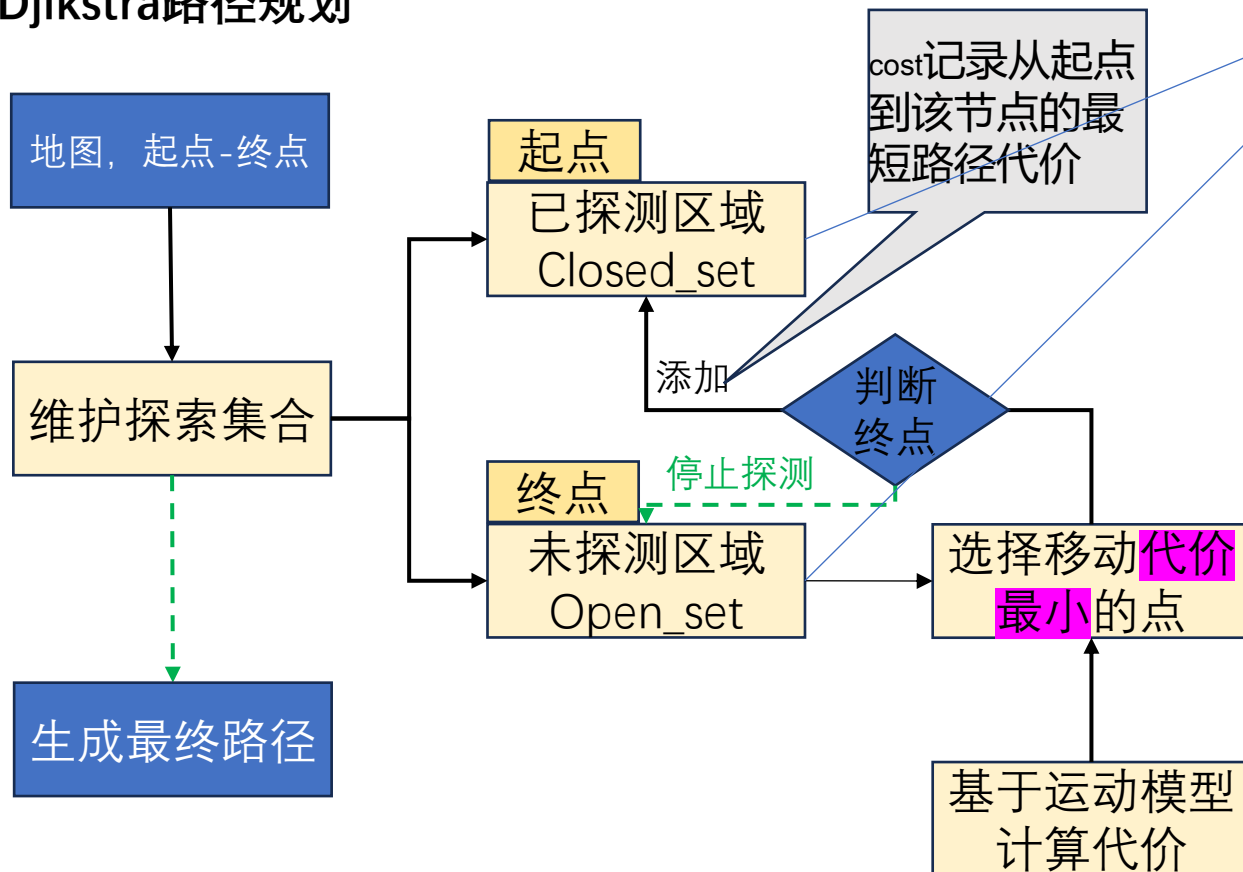
# Part 5 | 软件环境准备

Python环境



## 路径规划-基于网格地图的Dijkstra

### Dijkstra路径规划



```
class Node:
    解释 | 添加注释 | X
    def __init__(self, x, y, cost, parent_index):
        self.x = x
        self.y = y
        self.cost = cost
        self.parent_index = parent_index
```

两个集合均采用Node数据结构来保存栅格地图但是起到了类似链表的功能

```
node = self.Node(current.x + move_x,
                  current.y + move_y,
                  current.cost + move_cost, c_id)
```

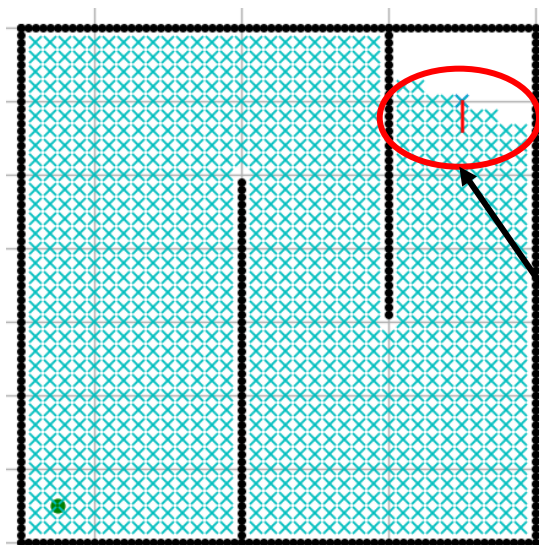
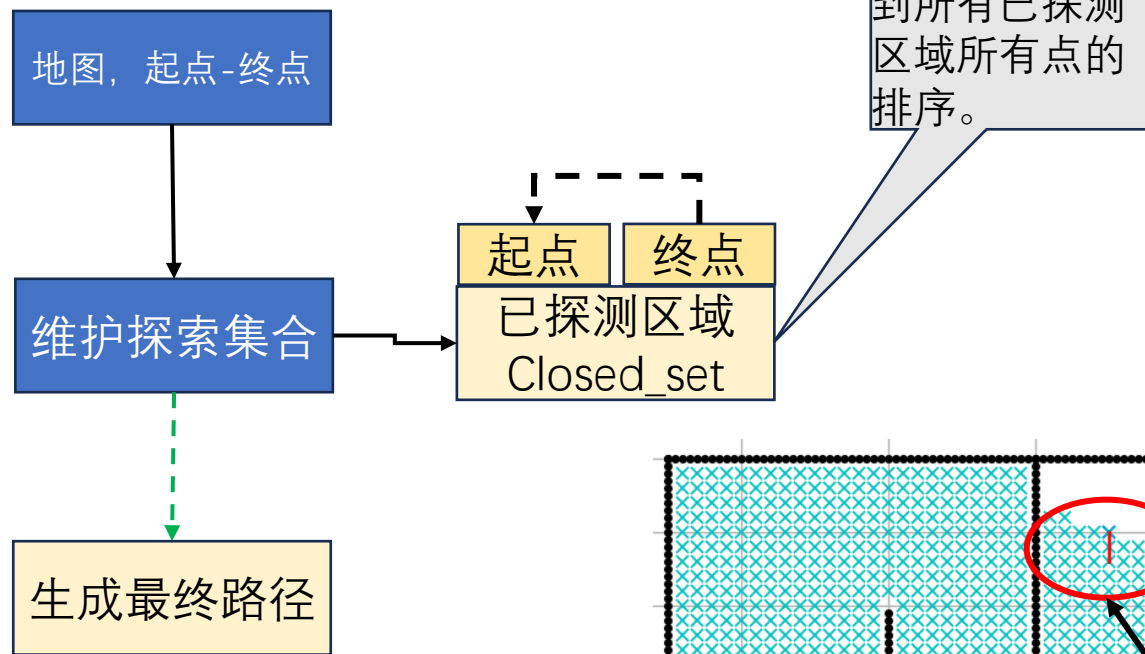
# Part 5 | 软件环境准备

Python环境



## 路径规划-基于网格地图的Dijkstra

### Dijkstra路径规划



```
def calc_final_path(self, goal_node, closed_set):  
    # 初始化路径列表, 首先加入目标节点的坐标  
    rx, ry = [self.calc_position(goal_node.x, self.min_x)], [  
        self.calc_position(goal_node.y, self.min_y)]  
  
    # 获取目标节点的父节点索引 (指向路径中的前一个节点)  
    parent_index = goal_node.parent_index  
  
    # 循环回溯父节点链, 直到找到起点 (parent_index == -1)  
    while parent_index != -1:  
        # 从closed_set中获取父节点对象  
        n = closed_set[parent_index]  
  
        # 将父节点的坐标添加到路径列表中  
        rx.append(self.calc_position(n.x, self.min_x))  
        ry.append(self.calc_position(n.y, self.min_y))  
  
        # 更新父节点索引, 继续向上回溯  
        parent_index = n.parent_index  
  
    # 返回路径列表 (注意: 此时路径是从终点到起点的顺序, 需要在调用处反转)  
    return rx, ry
```

#### Universal Optimality of Dijkstra via Beyond-Worst-Case Heaps

BERNHARD HAEUPLER, INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria and ETH Zurich, Switzerland  
RICHARD HLADÍK, ETH Zurich, Switzerland and INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria  
VÁCLAV ROZHON, Charles University, Czech Republic and INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria  
ROBERT E. TARJAN, Princeton University, USA  
JAKUB TĚTEK, INSAIT, Sofia University "St. Kliment Ohridski", Bulgaria

In this paper we prove that Dijkstra's shortest-path algorithm, if implemented with a sufficiently efficient heap, is universally optimal in its running time, and with suitable small additions is also universally optimal

# Part 5 | 软件环境准备

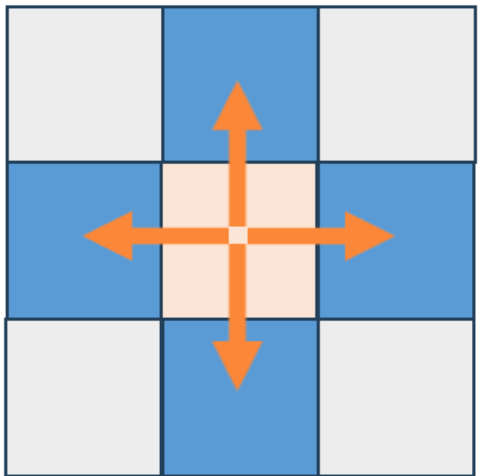
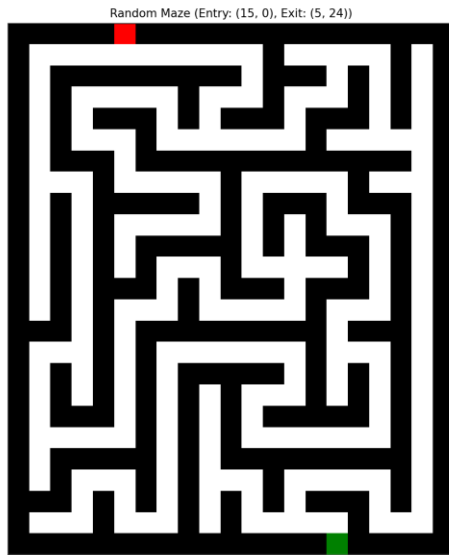
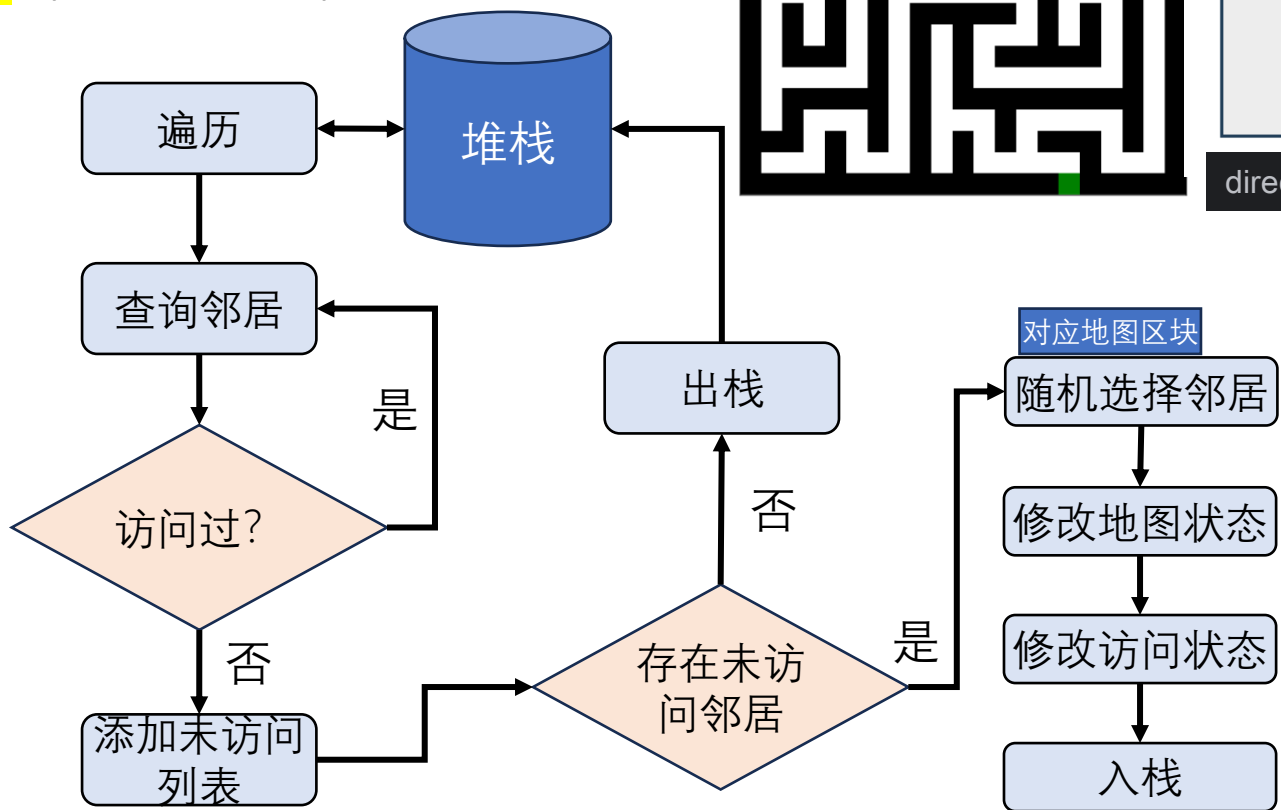
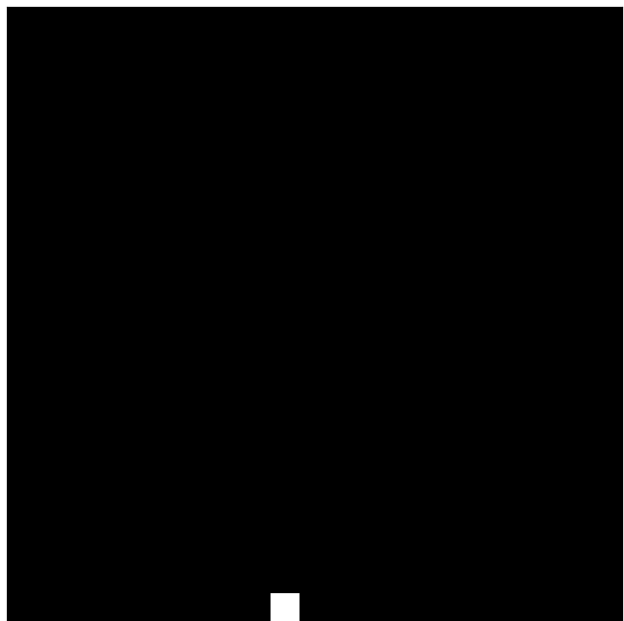
Python环境



## 路径规划-基于网格地图的Dijkstra

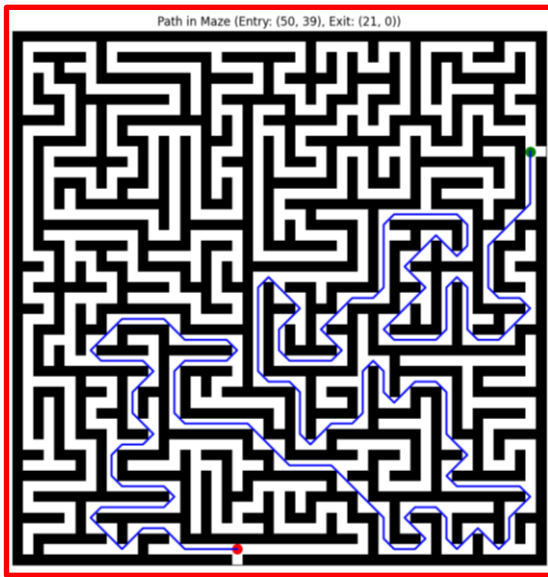
### 迷宫扩展

地图的一个极端就是空白，可以随便走；  
对应的另一个极端就是**只有墙**（外加出/入口）。



迷宫随机移动模型

directions = [(-2, 0), (0, 2), (2, 0), (0, -2)]



实际上是基于深度优先的搜索

实验要求：对随机迷宫进行Dijkstra路径搜索

# Part 5 | 软件环境准备

Python环境

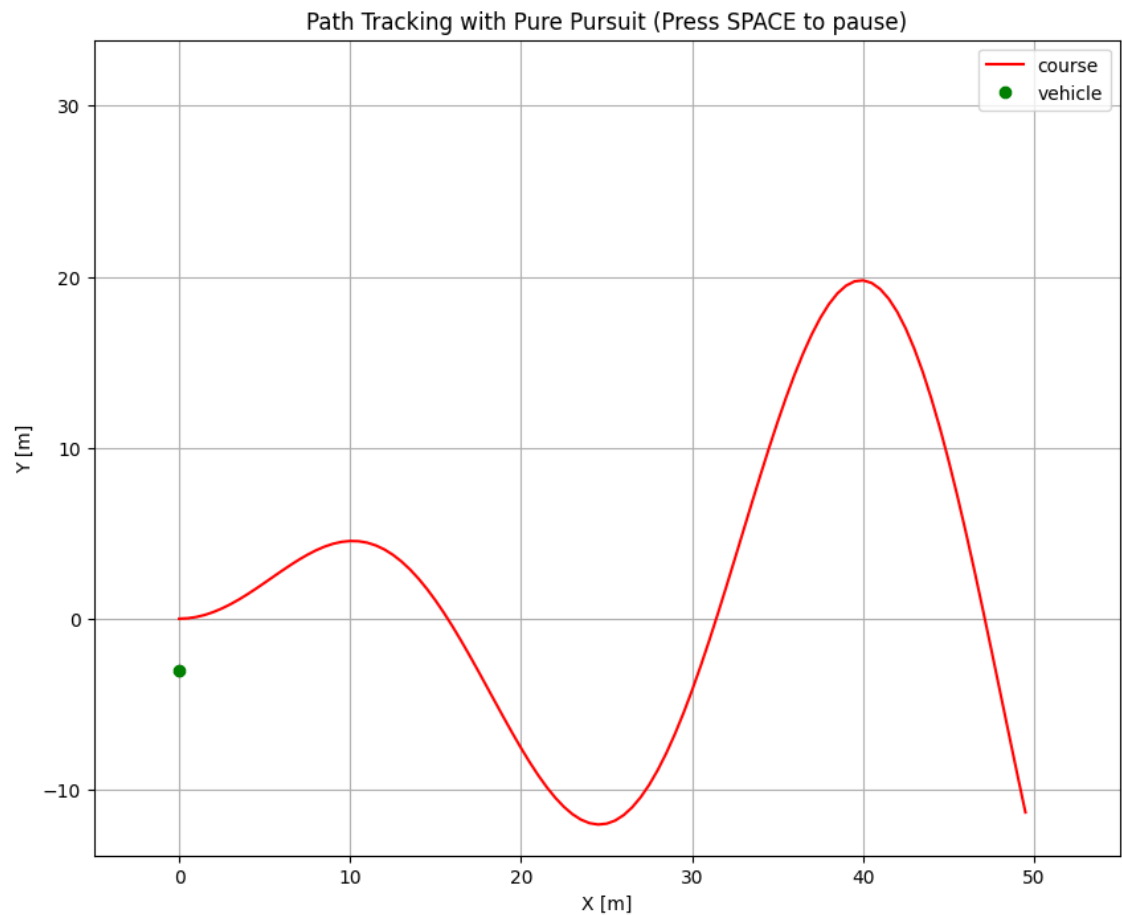


## 寻迹/路径跟踪

- 定位：粒子滤波算法
- 建图：光线投射网格地图
- 路径规划：基于网格地图的Dijkstra
- **寻迹：纯追踪跟踪 (Pure Tracking)**
  - 点击运行按钮，matplotlib绘制并实时更新算法结果。
  - **红色：**规划轨迹。
  - **绿色X：**目标点
  - **蓝色：**控制车辆 + 实际轨迹。

前面的示例，为了突出对应模块的算法本身，对Agent的运动进行简化，仅考虑质点的运动坐标、速度和航向+控制噪声。

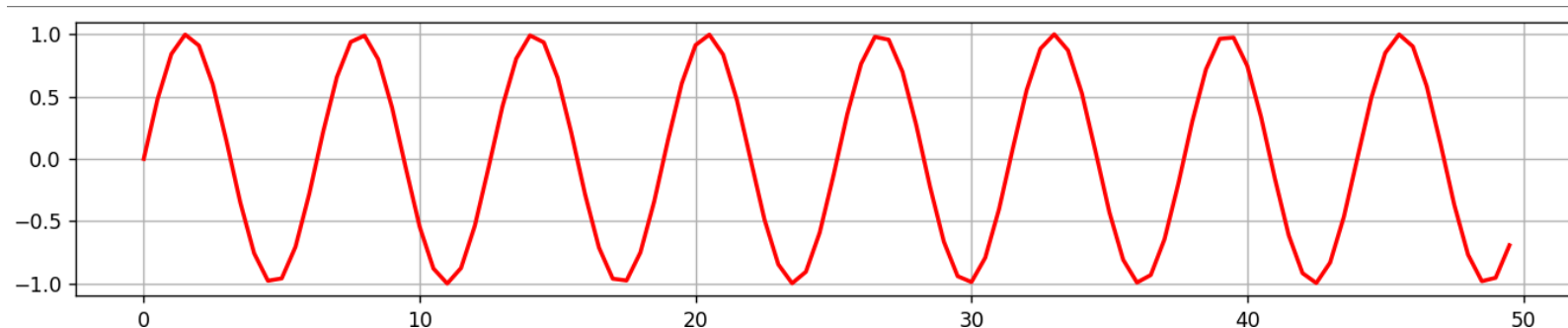
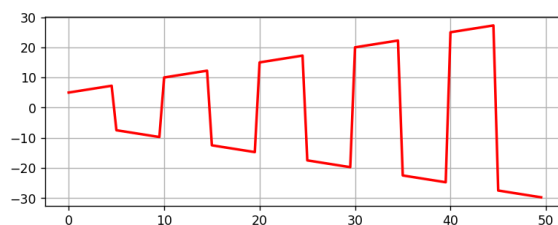
但寻迹任务中，运动体的动力学和运动学模型，是限制其**运动能力**的最大要素，需要进行建模。



# Part 5 | 软件环境准备

## 寻迹/路径跟踪

- 目标轨迹：定义一条从原点出发的正弦曲线。



```
cx = -1 * np.arange(0, 50, 0.5) if is_reverse_mode else np.arange(0, 50, 0.5)
cy = [math.sin(ix) for ix in cx]
```

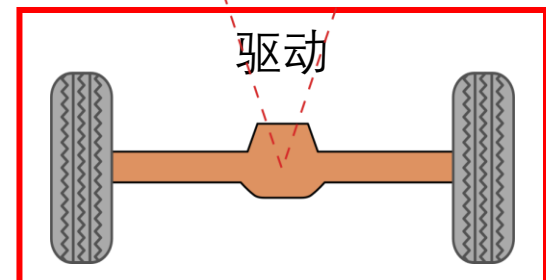
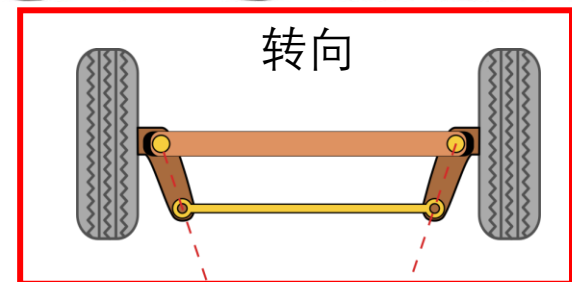
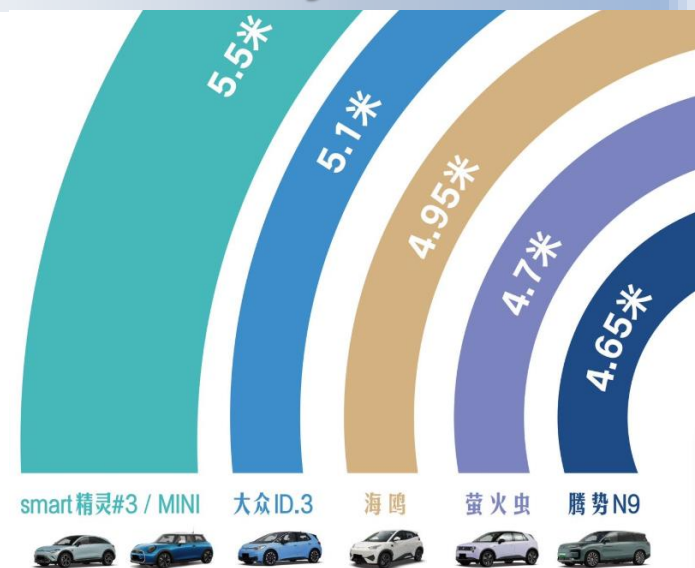
理想情况下：希望车辆能够从起点沿轨迹移动到终点。

实际情况：车辆受自身动力学因素限制，其行动能力并不是无限的。除了速度以外，车辆的尺寸，会影响车的运动学属性，进而限制任意运动。

比较著名的有：自行车模型（2轮），三轮模型和四轮模型。  
考虑的轮越多，计算公式越复杂，估计越精准。

## Python环境

越大当然得房率越高  
转向半径越小越灵活



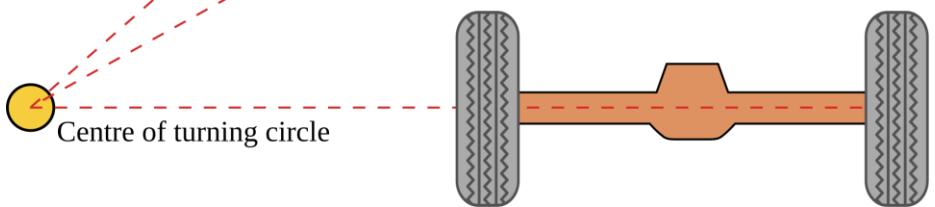
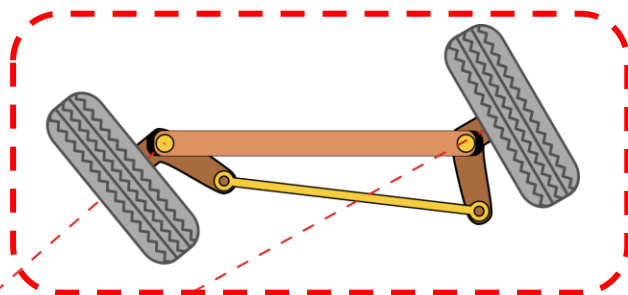


# Part 5 | 软件环境准备



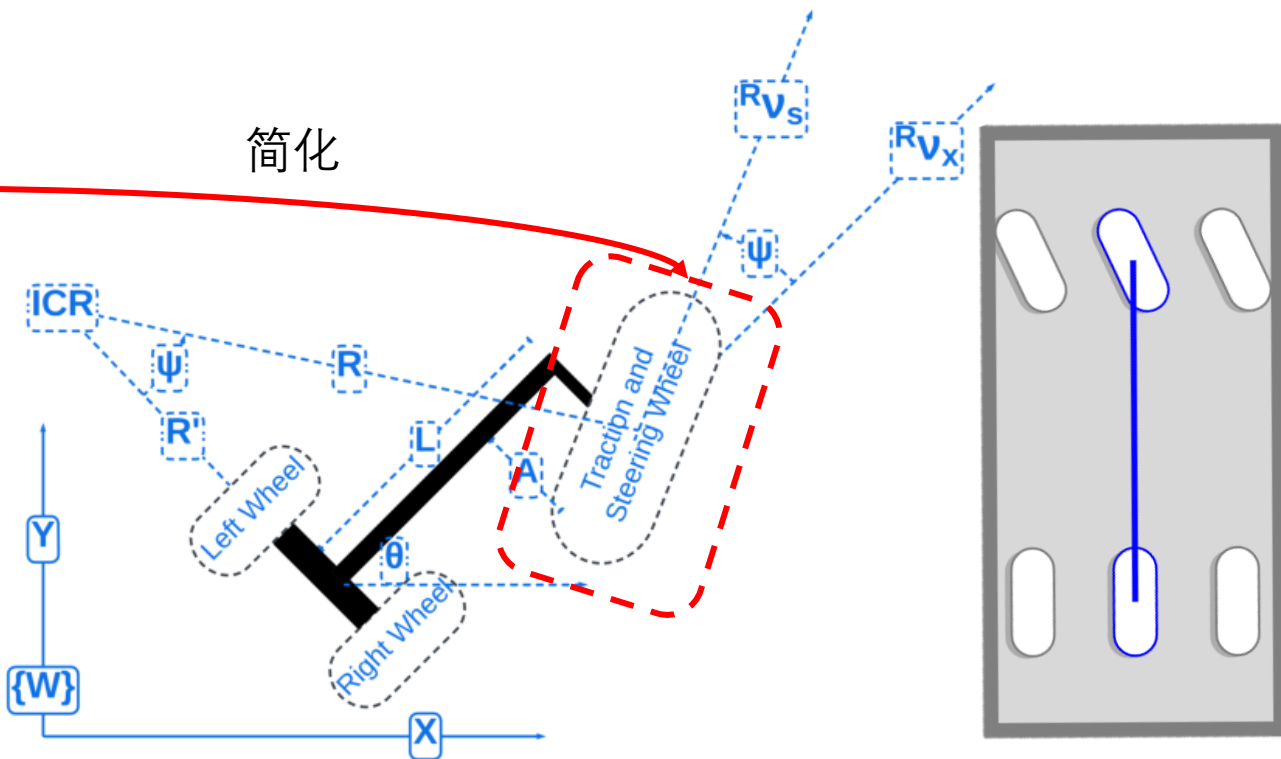
## 寻迹/路径跟踪

- 车辆运动学模型



阿克曼四轮模型中，两个前轮的转向半径不同  
内大外小，最接近真实情况。

简化



简化四轮模型中的横向控制，  
就是三轮车模型

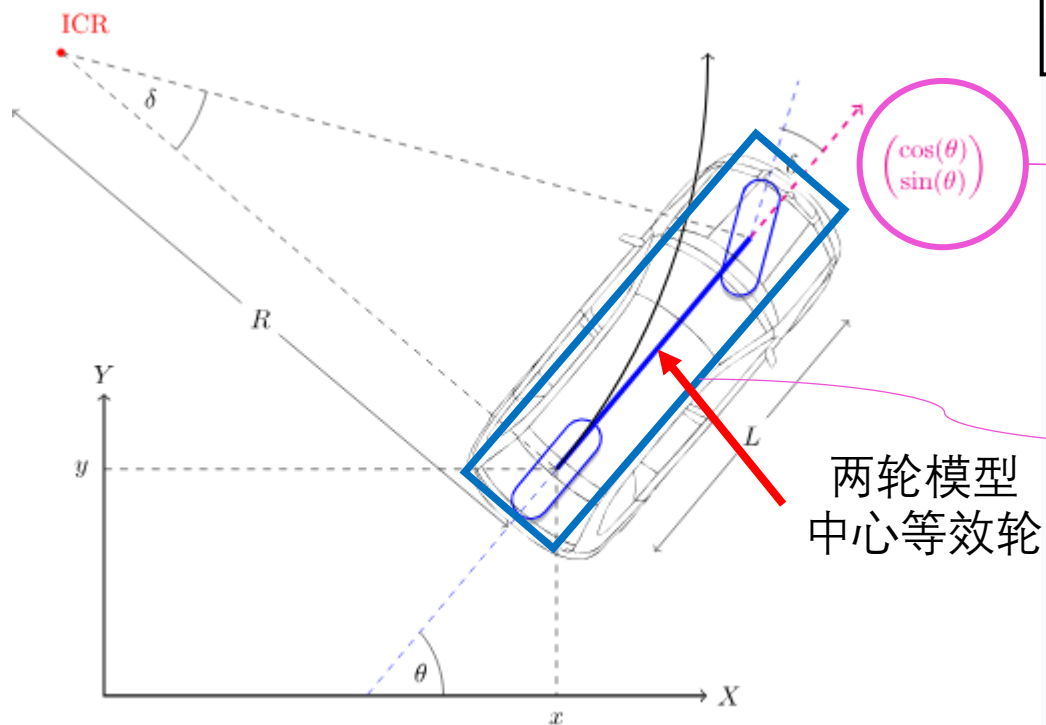
进一步简化后轮间距  
得到自行车模型  
非常接近之前的质点模型

# Part 5 | 软件环境准备

Python环境



寻迹/路径跟踪



每个时间片下的车辆运动状态存入States列表

```
class State:
```

```
def __init__(self, x=0.0, y=0.0, yaw=0.0, v=0.0, is_reverse=False):
```

```
self.x = x
```

```
self.y = y
```

```
self.yaw = yaw
```

```
self.v = v
```

```
self.direction = -1 if is_reverse else 1 # Direction based on reverse flag
```

```
self.rear_x = self.x - self.direction * ((WB / 2) * math.cos(self.yaw))
```

```
self.rear_y = self.y - self.direction * ((WB / 2) * math.sin(self.yaw))
```

```
def update(self, a, delta):
```

```
self.x += self.v * math.cos(self.yaw) * dt
```

```
self.y += self.v * math.sin(self.yaw) * dt
```

位置更新

```
self.yaw += self.direction * self.v / WB * math.tan(delta) * dt
```

速度更新

```
self.yaw = angle_mod(self.yaw)
```

航向角更新

```
self.v += a * dt
```

```
self.rear_x = self.x - self.direction * ((WB / 2) * math.cos(self.yaw))
```

```
self.rear_y = self.y - self.direction * ((WB / 2) * math.sin(self.yaw))
```

轴距L=2.9m

车长=3.9m # 前后个加0.5

车宽=2m

车轮直径=0.6

车轮宽度=0.2

# Part 5 | 软件环境准备

Python环境

## 寻迹/路径跟踪

车辆控制:

- 横向=转向角度 # 航向角的变化
- 纵向=加速度 # 速度的变化

设目标速度10km/h=10/3.6 (m/s)

车辆的运动学特性建模:

- 最大转向半径  $\delta = \pi/4$  # 车轮的最大转向角度
- 速度比例增益系数  $K_p=1.0$  # P控制器

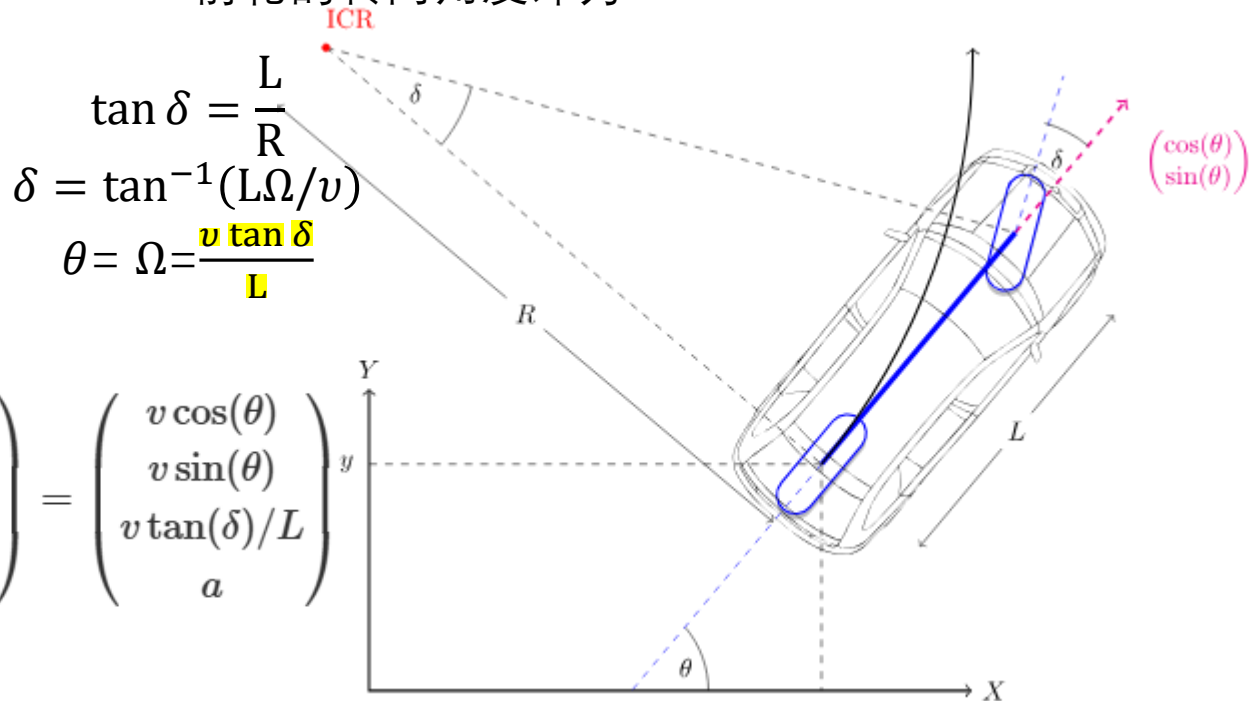
对于加速度, 使用简单的P控制器 (比例控制器), 基于当前速度和目标速度的差进行更新, 并且假设执行率100%, 调节 $K_p$ 可以模拟比例增益 (实际为PID控制的简化)。

```
def proportional_control(target, current):  
    a = Kp * (target - current)  
    return a
```

$A = \pi r^2$  其他类型控制器可以进行更复杂和精确的测量。

对于航向角 (仅考虑XOY平面), 刚体的车辆存在一个瞬时转动中心ICR, 一般用来作为车辆运动分析的参考点。

以两轮车轮模型为例, 车辆实际与以瞬时转动中心为圆, 后轮中心为切点, 进行切向运动。前轮的转向角度即为:



$$\tan \delta = \frac{L}{R}$$
$$\delta = \tan^{-1}(L\Omega/v)$$
$$\theta = \Omega = \frac{v \tan \delta}{L}$$

$$\frac{d}{dt} \begin{pmatrix} x \\ y \\ \theta \\ v \end{pmatrix} = \begin{pmatrix} v \cos(\theta) \\ v \sin(\theta) \\ v \tan(\delta)/L \\ a \end{pmatrix}$$

\*前轮以角速度绕后轮旋转

# Part 5 | 软件环境准备

Python环境



## 寻迹/路径跟踪

- Pure Tracking纯追踪

在纯追踪法中，会在目标轨迹上确定一个目标点(TP)，该目标点与车辆之间的距离为预瞄距离（ $L_{fc}=2$ ）。

搜索车辆中心到目标轨迹最小距离点

更新最近点

基于速度更新预瞄距离

沿路径查找超过育苗点的点

通过速度和增益动态更新 预瞄距离  
高速的时候看得更远，转向半径平缓  
低速看的相对近，对路径变化敏感  
防止震荡、超调

```
Lf = k * state.v + Lfc # update look ahead distance
```

```
while Lf > state.calc_distance(self.cx[ind], self.cy[ind]):  
    if (ind + 1) >= len(self.cx):  
        break # not exceed goal  
    ind += 1
```

```
def search_target_index(self, state):  
    if self.old_nearest_point_index is None:  
        dx = [state.rear_x - icx for icx in self.cx]  
        dy = [state.rear_y - icy for icy in self.cy]  
        d = np.hypot(dx, dy)  
        ind = np.argmin(d)  
        self.old_nearest_point_index = ind  
    else:  
        ind = self.old_nearest_point_index  
        distance_this_index = state.calc_distance(self.cx[ind],  
                                                  self.cy[ind])  
  
        while True:  
            distance_next_index = state.calc_distance(self.cx[ind + 1],  
                                                       self.cy[ind + 1])  
  
            if distance_this_index < distance_next_index:  
                break  
  
            ind = ind + 1 if (ind + 1) < len(self.cx) else ind  
            distance_this_index = distance_next_index  
        self.old_nearest_point_index = ind
```

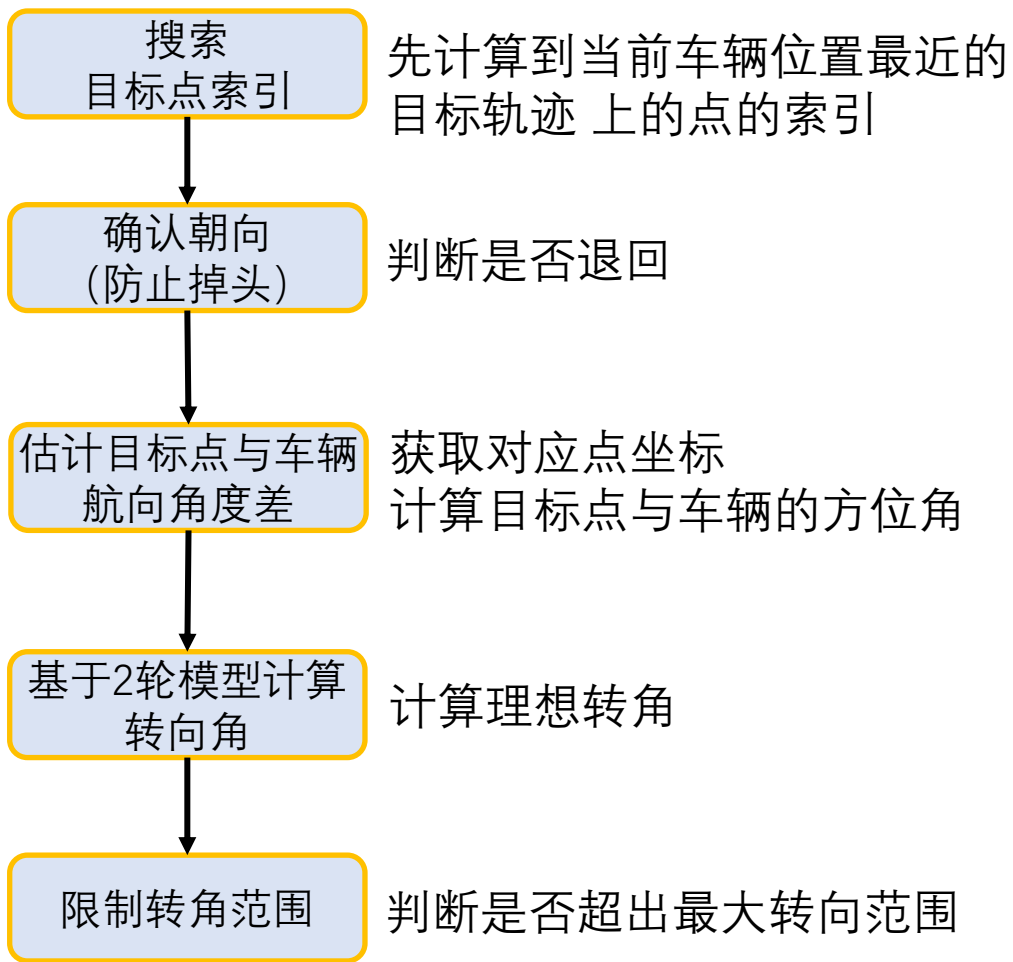
# Part 5 | 软件环境准备

Python环境



## 寻迹/路径跟踪

- 横向控制



```
def pure_pursuit_steering_control(state, trajectory, pind):
    ind, Lf = trajectory.search_target_index(state)

    if pind >= ind:
        ind = pind

    if ind < len(trajectory.cx):
        tx = trajectory.cx[ind]
        ty = trajectory.cy[ind]
    else:
        tx = trajectory.cx[-1]
        ty = trajectory.cy[-1]
        ind = len(trajectory.cx) - 1

    alpha = math.atan2(ty - state.rear_y, tx - state.rear_x) - state.yaw

    delta = state.direction * math.atan2(2.0 * WB * math.sin(alpha) / Lf, 1.0)

    delta = np.clip(delta, -MAX_STEER, MAX_STEER)

    return delta, ind
```

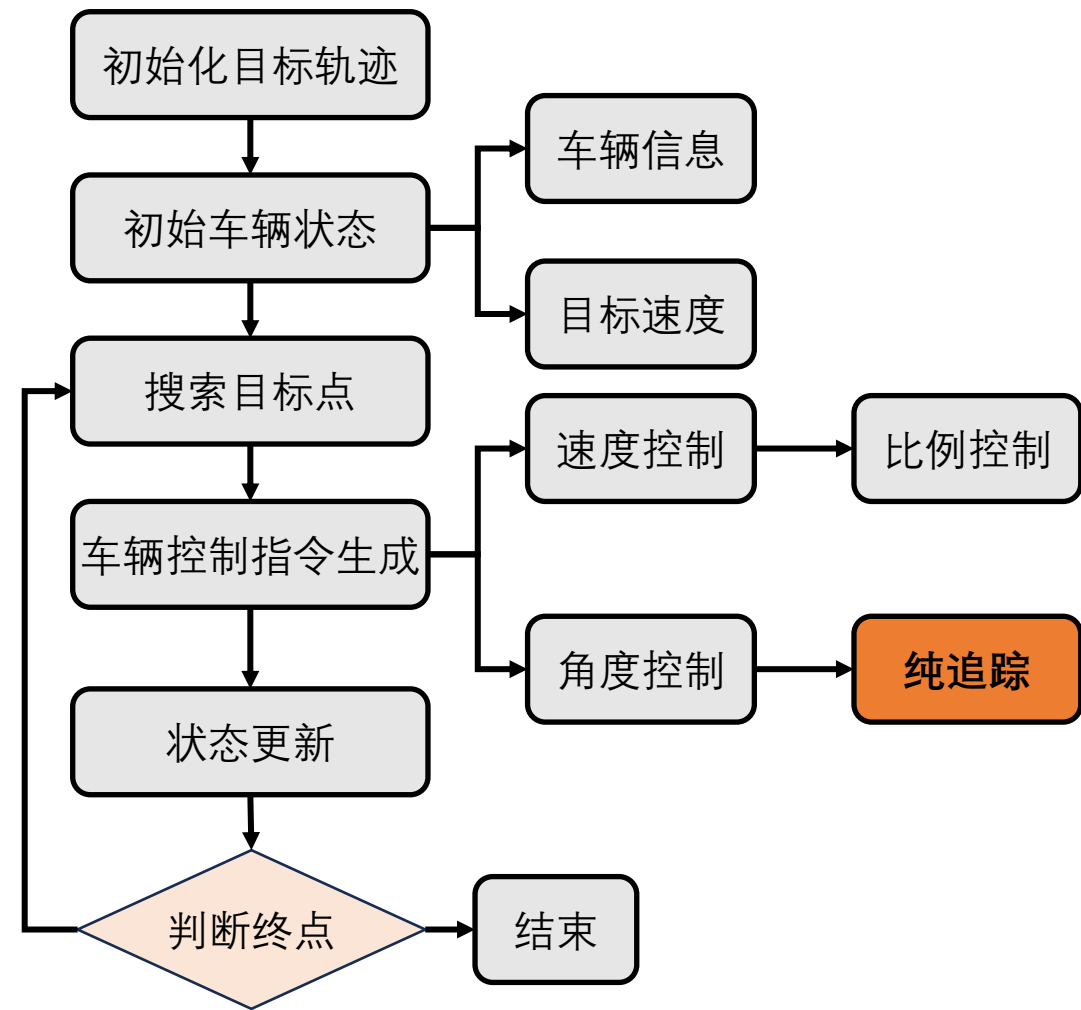
# Part 5 | 软件环境准备

Python环境



## 寻迹/路径跟踪

### • 总体流程



实际上pure tracking在整个框架中仅在角度（横向）控制使用算法。



# Part 5 | 课程考核

## 》 作业提交形式

1. 提交作业到邮箱：[21371058@buaa.edu.cn](mailto:21371058@buaa.edu.cn)。
2. 作业包含：a)代码、b)可视化结果、c)与大模型交互的对话过程  
(大多数AI工具包含分享功能，但注意提交后不要删除相应对话，否则会导致提交链接失效 (无法打开视为提交无效)，另一种折中方案为另存为可保存在本地的文件形式)。
3. 以单个算法为单元，对item2的内容进行打包，特别包括交互内容。
4. 注意标注学号+姓名。如：学号\_姓名\_课程1-作业1:轨迹算法1-xxxx.zip。
5. 每节课可以提交关于本节课内容上的优化建议，但不建议大于500字。若超过此字数，可使用AI进行压缩。被采纳可酌情加分。
6. 有问题**优先**咨询AI，多思考（代码只是一组分开都认识，合起来就不知道在说什么的符号而已），**其次**咨询助教。
7. **每次课作业有效时间不晚于下一次课前**：如第一次课10月24日，第一次作业提交不晚于10月30号00:00。
8. 成绩会基于所有人提交的所有内容进行**综合评估**，**提交实验末位5%的同学将不及格**。

注意ppt中的如下**作业指派信息**

噪声

**实验要求：**调整定位锚点位置（改变分布），模拟不同移动轨迹，如双曲线、椭圆、S形等机器人运动轨迹；并分析粒子采样策略、噪声分布对算法的影响。

| 物理意义 | 在代码中的使用 |
|------|---------|
|      |         |

**实验要求：**尝试修改雷达参数观测估计的结果。

- 但雷达角分辨率过小，而车辆的轮廓插值密度相对的大；则会出现 lesson1 中的穿透现象。**是否能够采用线段焦点/其它的方式修正？**
- 测距噪声则会导致估计错误。**是否可以通过引入粒子/Kalman滤波进行压制？**

**实验要求：**尝试在真实数据上 000004.bin 进行车辆检测。  
提示：如何消除地面点云干扰（问问gpt）？

# 开始实验



北京航空航天大学  
BEIHANG UNIVERSITY